

DEAKIN UNIVERSITY

SOFTWARE ARCHITECTURE AND SCALABILITY FOR INTERNET-OF-THINGS

ONTRACK SUBMISSION

Module 3 Learning Evidence

Submitted By:

Jahan GARG

s226100103

2026/08/31 20:59

Tutor:

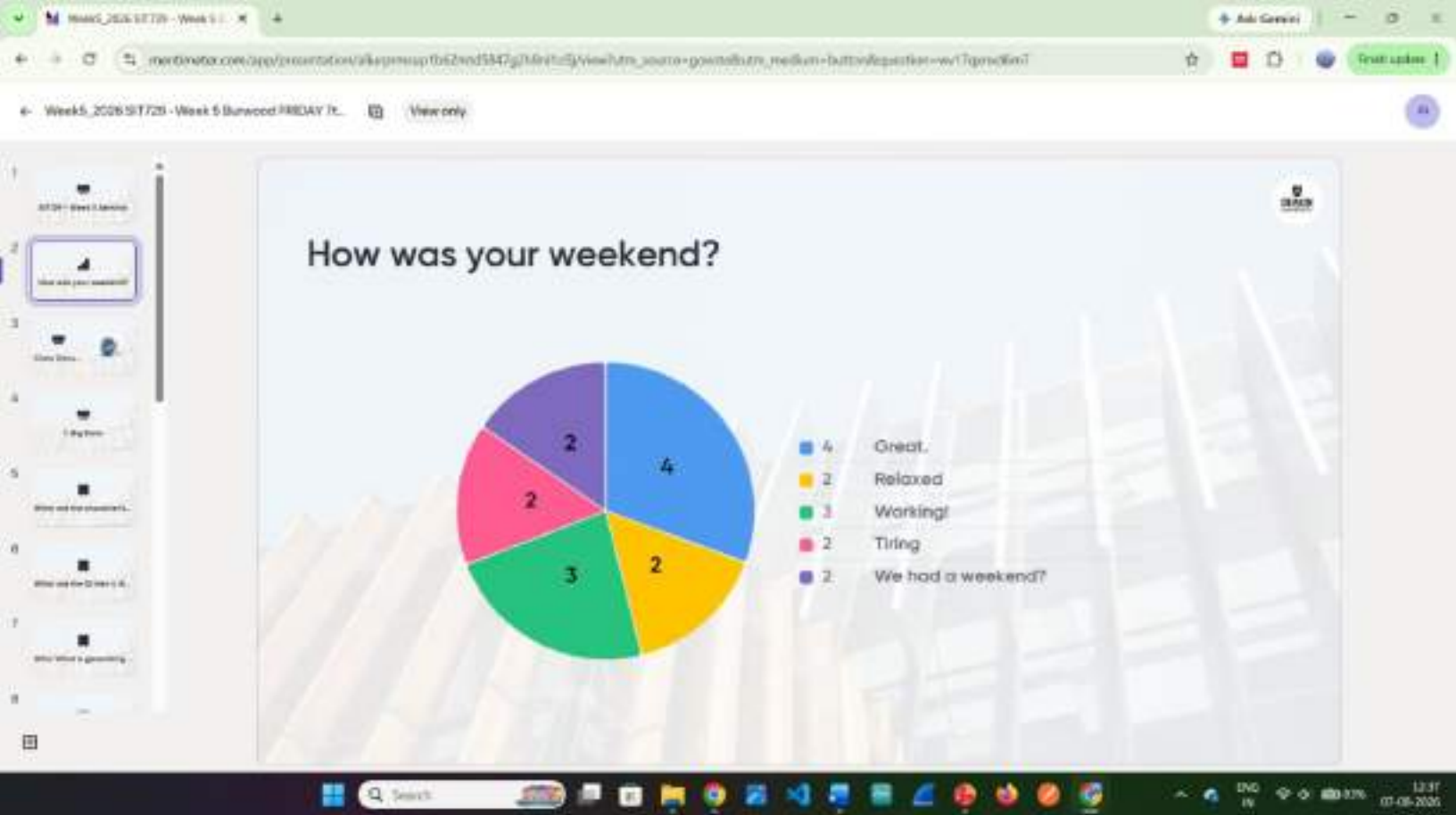
Nisha KUMARI

Task Outcomes		Supports
TLO1	Demonstrate engagement with the following unit learning outcomes (legacy)	ULO1 Demonstrate ULO2 Apply ULO3 Develop ULO4 Deploy

August 31, 2026



SIT729 - Week 5 Seminar



3
4
5
6
7
8

3720 - Week 5 Admin

Your data plan exceeded!

Class Status

0 Big Data

What will data processing...

What will the 3D look li...

What will be generating...

Class Discussion

1. Big Data.
2. Big Data and Cloud Computing.
3. Data Warehousing



Week5_2026 SIT20 - Week 5 L1 - Week 5 L1 - Week 5 L1

meritxeta.com/app/promotion/valignupfb62nd5347g7hfrtc5viewutm_source=google&utm_medium=button&utm_campaign=esg2026

Week5_2026 SIT20 - Week 5 Burwood FRIDAY 11:00 AM View only

1. Big Data

12:37 07-06-2026

What are the characteristics of Big Data?



What are the (3 then 4 then) Six V's of Big Data?

A word cloud featuring various words starting with the letter 'V'. The words are arranged in a cluster, with 'volume' and 'variability' at the top, 'value' and 'veracity' in the middle, 'variety' and 'velocity' at the bottom, and 'validity', 'volatility', and 'visualization' scattered around. The words are in different colors and sizes, creating a dynamic visual effect.

Week5_2026 SIT720 - Week 5

meritdata.com.au/promotional/askprinsupfb62nd5347g7h6rftc5/view.htm_source=govmellum_redeem-button&partner=afftr05fa

Week5_2026 SIT720 - Week 5 Burwood FRIDAY 11... View only

1

SIT720 - Week 5 Learning

2

Your role (you're responsible)

3

Course Overview

4

1 Big Data

5

What are the components...

6

What are the 3D data is...

7

What is data is generating

8

2. Big Data and Cloud Computing.

12:37

07-08-2026

What is NoSQL?

The absence of SQL

Non-relational database

non-relational databases

approach to database design based on key-value

no relational database

Not Only SGA

NoSQL is a type of database that does not use the traditional fixed rows, columns or tables structure of relational databases.

non-relational databases that store data in a non-tabular format

NoSQL is a non-relational database designed to store, manage, and process large volumes of unstructured or semi-structured data with high scalability and flexibility.

non-relational databases such as mongodb

flexible database system that stores different types of data without

NaClO is antiseptic that cleans

Week5_2026 SIT729 - Week 5.1 - < >

monday.com/app/promotion/alkyrmupfb62nd5347g7h6rhc5/viewutm_source=google&utm_medium=button&gclid=6c4pff6c0dgn

☆ ⓘ 🔒 ⚙️

Trust updates

Week5_2026 SIT729 - Week 5 Burwood FRIDAY 11.00 AM

View only

1 Day later

What are the characteristics...

What are the 10 characteristics...

What time is generating...

1 Day later and Cloud 10...

What are the 10 characteristics...

What are the 10 characteristics...

What are the characteristics and advantages of using NoSQL for scalable IoT applications?

Flexible

Easy structure and highly scalable

Flexible schema

Fast read and write performance

High scalability, flexible schema, fast performance, distributed architecture, fault tolerance, and real-time data handling.

can store various sensor's data in same collections

high performance

Horizontal Scalability

Highly scalable and flexible and time saving

Store data across multiple nodes for high availability and fault tolerance.

handles large volume of data

handles different types of IoT data, and even single device multiple

Windows Taskbar

12:39 07-08-2026

3. Data Warehousing.

Week5_2026 SIT219 - Week 5

Week 5 Burwood FRIDAY 11

View only

7

What is a gateway?

8

2 Big Data and Cloud Co.

9

Amazon AWS CloudWatch

10

What is a data lake?

11

What are the challenges?

12

3 Data Warehousing

13

What is Data Warehousing?

14

Why might we need Data Warehousing for building scalable IoT applications?

To store huge chunks of data in one place

Ability to compare historical data to current input

Lots of data to store

store large amount of data

Records user history

To process and store a lot of data

Centralizes IoT data for scalable storage, fast analysis, and better decision-making.

reduces the stress on your own hardware, makes it easy to modularise scope

As a method of generating meaningful insights from disparate or unstructured sensor data.

Have lots of info to build of

so that the IoT data can be easily accessed for writing and reading

Archived data can be used to train AI

12:39

07-08-2026

SIT314/SIT729 – Week 5 Group Activity

Designing JSON for IoT Applications

Prepared by: Jahan Garg

Student ID: 226100103

Unit: SIT314/SIT729 – Software Architecture and Scalability for Internet-of-Things

Task: Week 5 Group Activity – Designing JSON for IoT Applications

Date: 30 August 2026, Week 5, Trimester 2, 2026

Overview

This activity was discussed collaboratively during the SIT314 seminar with my classmates Melissa Woo and Madhur Gupta, where different attribute choices and structural ideas for each of the seven scenarios were debated as a group. However, in keeping with OnTrack's individual submission requirement for module evidence, the schema designs, written analysis and referencing set out below are entirely my own independent work, reflecting my own understanding of the concepts covered in this unit.

The Week 5 Group Activity moves the unit's focus from network protocols and message delivery, which formed the basis of the Week 3 and Week 4 Technical Tasks, toward the structure of the payload that those protocols actually carry. JavaScript Object Notation, generally shortened to JSON, is a lightweight and text based data interchange format built from ordered field and value pairs, and it has become the de facto standard for representing sensor readings and command requests in Internet of Things systems because it is readable by a person while still being trivial for a machine to parse across almost any programming language [1]. As the unit's own Module 3 material on IoT and Big Data explains, a modern sensing application must capture, transmit and interpret data consistently across many heterogeneous devices, and a properly considered payload format is the first practical step toward that consistency once a system begins to scale beyond a handful of devices [2].

Before beginning this activity, I already had some hands on familiarity with embedded platforms from earlier personal project work, having previously used the Arduino Nano 33 IoT and the Arduino UNO R3 boards, although I had not yet had the opportunity to work directly with the ESP32 board referenced elsewhere in this unit's device platform material. This background gave me a reasonable starting point for reasoning about what a real embedded sensor or actuator could realistically report or accept, even though designing a JSON schema itself, rather than simply reading a sensor value in code, was a new exercise for me and is the central learning outcome of this particular activity.

Seven distinct scenarios were assigned for this task: a room temperature sensor, the status of a self driving car, the location of a person, a weather update for a location, the status of a smart vacuum cleaner, a television programme change request, and a request message to set an air conditioning setting. Each of the following sections presents the JSON schema I designed for that scenario,

together with a short block diagram showing where that payload sits within a wider publish and subscribe communication flow, and a written justification of every field included, grounded in the unit's own lecture material and in external, authoritative sources where appropriate.

Design Principles Applied Across All Seven Schemas

Before presenting the seven individual schemas, it is worth setting out the design principles that were applied consistently across every one of them, since a scalable Internet of Things deployment depends on predictable and uniform data structures rather than seven unrelated formats invented independently of one another [2]. Table 1 summarises these five principles and how each one was put into practice.

Principle	How it was applied in this activity
Unique device or entity identifier	Every payload carries an identifying field, such as <code>device_id</code> , <code>vehicle_id</code> or <code>user_id</code> , so that a subscriber receiving many messages on a shared topic can attribute each reading back to its source device with no ambiguity.
Timestamp in ISO 8601 format	Every payload includes a timestamp field written in ISO 8601 format, for example <code>2026-08-31T18:45:00+10:00</code> , which is the internationally recognised standard for exchanging date and time values between independent systems without confusion over time zone or ordering [3].
Explicit units embedded in field names	Numeric fields such as <code>temperature_celsius</code> or <code>speed_kmh</code> state their unit directly within the field name itself, removing any risk that a consuming application might silently assume the wrong unit system when parsing the value.
Considered use of flat and nested structures	Simple scalar readings are kept as flat fields for lightweight parsing, while a genuinely compound value, such as a geographic coordinate pair, is nested as its own sub object so that its internal structure is preserved rather than flattened into two unrelated looking fields.
Explicit null values for exceptional states	A field such as <code>error_code</code> is always present in its schema and is simply set to null when no fault exists, rather than being omitted from the payload entirely, so that a consuming application's parsing logic can rely on one consistent schema shape at all times.

Table 1. Design principles applied consistently across all seven JSON schemas in this activity.

Figure 1 below shows the generic four stage flow that each schema is intended to travel through in practice: a sensor or client device produces a reading or a request, that value is structured as a JSON payload, the payload is published to a shared MQTT broker under a suitably scoped topic, and a subscribing application or actuator finally consumes and acts on it. Each of the seven scenarios that follow simply adapts this same generic flow to its own particular source and destination, which is a direct extension of the publish and subscribe architecture already demonstrated using `broker.hivemq.com` in the Week 3 Technical Task [4].



Figure 1. Generic four-stage data flow shared by all seven scenarios in this activity.

1. A Room Temperature Sensor

Suggested MQTT topic: /jahan226100103/room/study2/temperature

```

{
  "device_id": "room_temp_sensor_01",
  "location": "Study Room 2, Building LA, Deakin Burwood",
  "timestamp": "2026-08-31T18:45:00+10:00",
  "temperature_celsius": 23.4,
  "humidity_percent": 46.2,
  "battery_level_percent": 87
}
  
```

This schema was designed to represent a fixed indoor temperature sensor, comparable to the smart home examples discussed in Module 1 of this unit [5]. The `device_id` field allows a building management dashboard to tell this sensor apart from every other room sensor publishing to the same shared broker, while the `location` field offers a plain, human-readable description that a facilities manager could use directly, without first needing to look up `device_id` in a separate device registry table. Humidity was included alongside temperature because most commercially available combined temperature and humidity sensors report both readings together within a single transmission cycle, and this pairing is consistent with the smart air conditioning scenario I analysed in my own Week 2 Group Activity submission [6]. The `battery_level_percent` field reflects the practical reality that a sensor of this kind is very likely to be battery powered rather than mains powered, and remote battery monitoring becomes an essential requirement once a deployment scales across many rooms, since nobody could reasonably inspect hundreds of individual sensors by hand on a regular basis.



Figure 2. Data flow designed for a room-temperature sensor scenario.

2. The Status of a Self-Driving Car

Suggested MQTT topic: /jahan226100103/vehicle/AV-2291/status

```
{
  "vehicle_id": "AV-2291",
  "timestamp": "2026-08-31T18:46:12+10:00",
  "location": {
    "latitude": -37.8480,
    "longitude": 145.1150
  },
  "speed_kmh": 42.5,
  "heading_degrees": 187,
  "autonomous_mode": true,
  "current_manoeuvre": "lane_keep",
  "obstacle_detected": false,
  "battery_level_percent": 68,
  "system_status": "nominal"
}
```

The self-driving car schema builds directly on the driverless taxi scenario I examined in my Week 2 Group Activity, which established that an autonomous vehicle's on-board computing unit must process high-throughput sensor data continuously and maintain ongoing health monitoring of both its hardware and its software [6]. The location field was nested as a latitude and longitude pair rather than left as two flat fields, since a geographic coordinate is conceptually one single compound value rather than two independent readings, and this follows the same point representation convention defined in the GeoJSON specification [7]. The heading_degrees and current_manoeuvre fields were included because a fleet control centre supervising many vehicles at once needs to know not only where a vehicle currently is, but also what it is doing and in which direction it is heading, information that becomes essential when several autonomous vehicles must be coordinated safely through the same intersection. The system_status field uses a small, closed set of values such as nominal, degraded or critical, mirroring the health status pattern I had already established in the topic hierarchy for the Vehicle Asset Health scenario in my Week 3 Technical Task [4].

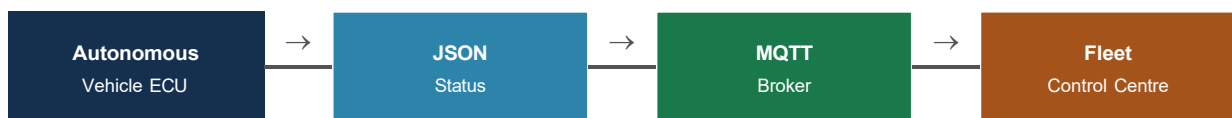


Figure 3. Data flow designed for the status of a self-driving car scenario.

3. The Location of a Person

Suggested MQTT topic: /jahan226100103/user/4471/location

```
{
  "user_id": "user_4471",
  "timestamp": "2026-08-31T18:47:03+10:00",
  "latitude": -37.8478,
  "longitude": 145.1148,
  "altitude_m": 58.2,
  "accuracy_m": 5.0,
  "speed_mps": 1.3,
  "location_source": "gps"
}
```

Unlike the vehicle status schema, this payload represents a smartphone reporting the location of the person carrying it, in the same spirit as the Google Maps Traffic crowdsourcing example discussed under Module 1.7 as a genuinely massive scale IoT application [8]. The `accuracy_m` field was included because a GPS derived position always carries a margin of error, and any consuming application, such as a location sharing service, needs that margin available before it displays a person's position to somebody else with unwarranted confidence. The `location_source` field allows the very same schema to be reused when a position is instead calculated through Wi-Fi triangulation or cell tower positioning, both of which typically produce a noticeably larger `accuracy_m` value, without needing an entirely separate payload structure for every possible positioning technique.



Figure 4. Data flow designed for the location of a person scenario.

4. A Weather Update for a Location

Suggested MQTT topic: /jahan226100103/weather/melbourne_cbd

```
{
  "location": {
    "city": "Melbourne",
    "country": "AU",
    "latitude": -37.8136,
    "longitude": 144.9631
  },
  "timestamp": "2026-08-31T18:00:00+10:00",
  "temperature_celsius": 14.8,
  "feels_like_celsius": 13.1,
  "humidity_percent": 72,
  "wind_speed_kmh": 18.4,
  "wind_direction_degrees": 210,
  "pressure_hpa": 1008,
  "condition": "light_rain"
}
```

This schema follows the general field naming pattern used by established commercial weather providers such as OpenWeatherMap, which reports temperature, humidity, wind and a short textual condition summary together as one combined update rather than as several separate messages [9]. This design choice is also consistent with the forest fire monitoring scenario I analysed in my Week 2 Group Activity, where temperature, humidity and smoke readings were identified as needing to be captured and timestamped together specifically to allow later correlation between them [6]. The `feels_like_celsius` field, kept distinct from the raw `temperature_celsius` reading, reflects the fact that wind and humidity meaningfully affect how a temperature is actually perceived, and reporting both figures allows a downstream application to display whichever value is most relevant to its intended audience.

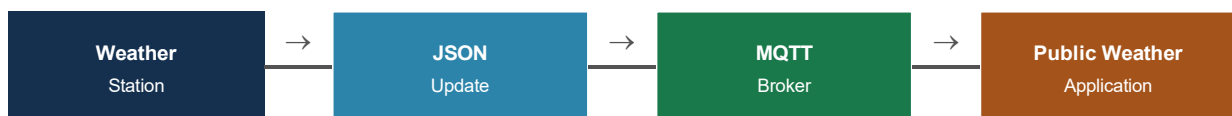


Figure 5. Data flow designed for a weather update for a location scenario.

5. The Status of a Smart Vacuum Cleaner

Suggested MQTT topic: /jahan226100103/vacuum/bot07/status

```
{
  "device_id": "vacuum_bot_07",
  "timestamp": "2026-08-31T18:50:41+10:00",
  "cleaning_status": "cleaning",
  "current_room": "living_room",
  "battery_level_percent": 54,
  "bin_full": false,
  "cleaning_mode": "auto",
  "total_area_cleaned_m2": 32.5,
  "error_code": null
}
```

The smart vacuum cleaner schema reflects the home integration platform category discussed in the unit's Smart Platforms lecture, where a household robot of this kind is typically controlled through a broader consumer platform such as Samsung SmartThings or Google Home rather than through one narrow, standalone application [10]. The `error_code` field is deliberately present as an explicit null value rather than simply left out when no fault exists, since this means a consuming application's parsing logic only ever has to check whether the value is null or a specific fault string, such as `stuck` or `bin_removed`, instead of having to handle two entirely different payload shapes depending on whether a fault happens to be active. The `cleaning_mode` field allows this same status schema to describe both a routine automatic clean and a user requested spot clean without needing a second, separate schema purely for that distinction.

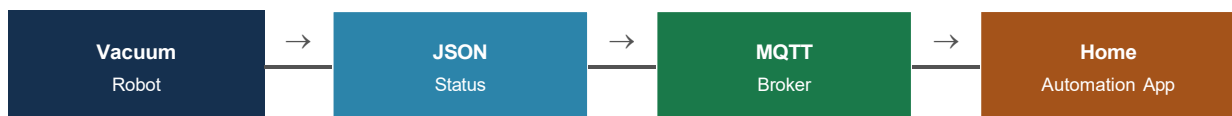


Figure 6. Data flow designed for the status of a smart vacuum cleaner scenario.

6. A Television Programme Change Request

Suggested MQTT topic: /jahan226100103/tv/living_room/command

```
{
  "device_id": "smart_tv_living_room",
  "command": "change_channel",
  "timestamp": "2026-08-31T18:52:15+10:00",
  "requested_channel_number": 2,
  "requested_channel_name": "ABC HD",
  "volume_level": 22,
  "requested_by": "mobile_app"
}
```

Unlike the previous sensor readings, this schema represents a command flowing in the opposite direction, from a controlling application down to an actuating device, mirroring the command branch of the topic hierarchy I designed for the Vehicle Asset Health scenario in the Week 3 Technical Task, where cloud to vehicle command topics such as command/derate were deliberately kept separate from the vehicle's own sensor reporting topics [4]. The explicit command field, set here to `change_channel`, allows the very same topic to eventually support other instruction types, such as `power_off` or `set_volume`, simply by changing the value of this one field, rather than requiring an entirely new topic for every possible instruction a smart television might receive. The `requested_by` field provides a simple audit trail identifying which client actually issued the command, which becomes genuinely useful in a household where several people, or even automated routines, may be capable of controlling the same television.

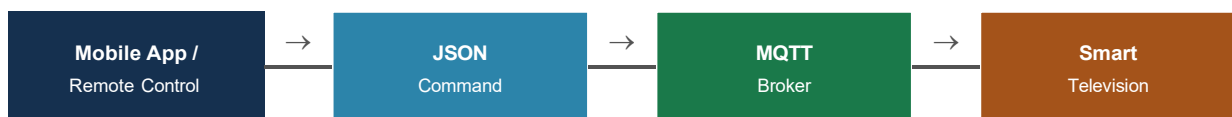


Figure 7. Data flow designed for a television programme change request scenario.

7. An Air-Conditioning Setting Request

Suggested MQTT topic: /jahan226100103/ac/bedroom/command

```
{
  "device_id": "ac_unit_bedroom",
  "command": "set_temperature",
  "timestamp": "2026-08-31T18:54:30+10:00",
  "target_temperature_celsius": 21.0,
  "mode": "cool",
  "fan_speed": "auto",
  "requested_by": "mobile_app"
}
```

This final schema completes the pairing suggested by the smart air conditioning scenario in my Week 2 Group Activity, where a building management system was identified as needing to interact reliably with damper and fan actuators through a tightly closed feedback loop [6]. The mode field, restricted to values such as cool, heat, fan_only or auto, together with the separate fan_speed field, allows one single command message to fully specify the desired operating state in a single publish action, rather than requiring three or four separate command messages to be sent and individually acknowledged in sequence. This design reduces the number of discrete MQTT publish events needed to change the unit's behaviour, which is a meaningful efficiency consideration when, as covered under Module 2.1, thousands of similar units could conceivably be issuing and receiving commands across one large commercial building at the same time [11].



Figure 8. Data flow designed for an air-conditioning setting request scenario.

Observations

Working through all seven scenarios in sequence made one pattern increasingly clear as the activity progressed. The five scenarios describing a sensor reading, namely the room temperature sensor, the self driving car, the person's location, the weather update and the vacuum cleaner status, all naturally gravitated toward the same underlying shape of an identifier, a timestamp and a set of measured values, even though I had not deliberately set out to enforce that similarity from the very beginning. The two remaining scenarios, the television programme change and the air conditioning setting, instead gravitated toward a second, quite different shape built around an identifier, an explicit command field and a set of requested target values. This was a genuinely useful observation, because it suggests that a properly scaled IoT platform does not need to write one bespoke parser for every single device type it supports, but could instead recognise these two broad payload families and process every device belonging to a given family with the same shared parsing logic.

Comparison Across the Seven Scenarios

Table 2 below compares all seven schemas by their primary direction of data flow, distinguishing sensor-originated telemetry, which travels from a device up to the broker and onward to an application, from command messages, which travel in the opposite direction from an application down to an actuating device. This distinction reflects the Sense, Think, Act loop introduced in the unit's Smart Platforms lecture, in which sensing components collect data, thinking components process that data, and actuation components put the resulting decisions into effect [10].

Scenario	Direction	Key fields	Typical update pattern
1. Room temperature	Sensor to broker	temperature_celsius, humidity_percent	Periodic, roughly every 60 seconds
2. Self-driving car status	Sensor to broker	location, speed_kmh, system_status	Continuous, sub-second interval
3. Person location	Sensor to broker	latitude, longitude, accuracy_m	Periodic, roughly every 5 to 10 seconds
4. Weather update	Sensor to broker	temperature_celsius, condition	Periodic, roughly every 10 to 60 minutes
5. Vacuum cleaner status	Sensor to broker	cleaning_status, battery_level_percent	Event driven, with periodic updates
6. Television programme change	Application to actuator	command, requested_channel_number	On demand, event driven only
7. Air-conditioning setting	Application to actuator	command, target_temperature_celsius	On demand, event driven only

Table 2. Comparison of direction, key fields and update pattern across the seven designed schemas.

Key Notes

A small number of points are worth restating concisely for clarity when this submission is reviewed. Every one of the seven schemas above deliberately embeds its measurement unit directly within the field name itself, for instance `temperature_celsius` rather than a bare field named `temperature`, which removes any need for a consuming application to separately look up or assume the correct unit before performing any calculation on the value. The suggested MQTT topic listed under each scenario continues to use the unique `jahan226100103` namespace prefix I first adopted for the Week 3 Technical Task, consistent with the cooperative broker sharing convention discussed throughout this unit's MQTT material, and this is not a registered identifier of any kind, simply a self-chosen convention to avoid colliding with other students publishing to the same shared public broker [4]. Finally, the two command schemas, for the television and the air conditioning unit, were designed so that a single publish action can fully specify the desired target state, rather than needing several separate, sequential command messages to achieve the same result.

Extra Effort

Beyond the unit's own W3Schools tutorial reference for JSON syntax, I independently reviewed the official GeoJSON specification, RFC 7946, published by the Internet Engineering Task Force, to confirm that the nested latitude and longitude structure I designed for the self driving car and person location scenarios followed an established, internationally recognised convention for representing point coordinates, rather than an arrangement I simply invented for convenience [7]. I also examined the publicly documented field structure of the OpenWeatherMap current weather API to ground the weather update schema in a real, widely used commercial precedent rather than a purely theoretical design, which gave me greater confidence that the fields I selected, such as `feels_like_celsius` alongside the raw temperature reading, reflect genuine industry practice rather than an arbitrary personal choice [9].

Reflection

Working through these seven scenarios reinforced that designing a JSON schema is not simply a matter of listing whatever data a sensor happens to produce, but requires deliberately deciding on naming conventions, unit disclosure, nesting depth and null handling before a single message is ever transmitted across a real network. Comparing the self-driving car's nested location object against the room temperature sensor's entirely flat structure made it clear that the correct level of nesting depends on whether a field is genuinely a compound value, such as a coordinate pair, or simply a collection of independent scalar readings that happen to originate from the same physical device.

Designing the two command schemas, for the television and the air conditioning unit, also clarified an architectural distinction that had been less visible to me during the earlier MQTT exercises completed for the Week 3 Technical Task, namely that a genuinely scalable application layer protocol has to accommodate both directions of communication within one consistent design

philosophy, rather than focusing solely on the sensor-to-cloud direction that tends to dominate introductory IoT examples. Discussing the attribute choices for each scenario with Melissa Woo and Madhur Gupta during the seminar was valuable in testing whether my initial field selections were genuinely necessary rather than merely convenient, and that discussion helped confirm that fields such as `accuracy_m` for a GPS location and `error_code` for the vacuum cleaner were not optional extras, but were in fact essential for any consuming application to behave sensibly under realistic conditions such as positioning uncertainty or an unexpected device fault. This is a pattern I intend to carry forward into my Distinction project later in this trimester, particularly when deciding how many optional fields a payload schema can reasonably carry before it becomes difficult to maintain consistently across many different device types.

References

- [1] ECMA International, "The JSON Data Interchange Syntax," ECMA-404, 2nd ed., 2017.
- [2] Deakin University, "3.1 IoT and Big Data," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 3, CloudDeakin, 2026.
- [3] International Organization for Standardization, "ISO 8601-1:2019, Date and time, Representations for information interchange," ISO, 2019.
- [4] J. Garg, "SIT314/SIT729 Week 3 Technical Task, Machine-to-Machine Communication with MQTT," Deakin University, 2026.
- [5] Deakin University, "1.2 Scalability in IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1, CloudDeakin, 2026.
- [6] J. Garg, "SIT314/SIT729 Week 2 Group Activity, Technologies for IoT Applications," Deakin University, 2026.
- [7] Internet Engineering Task Force, "RFC 7946, The GeoJSON Format," IETF, 2016. Online. Available: datatracker.ietf.org/doc/html/rfc7946
- [8] Deakin University, "1.7 Devices and Platforms for Scalable IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1, CloudDeakin, 2026.
- [9] OpenWeatherMap, "Current Weather Data API," OpenWeather Ltd, 2026. Online. Available: openweathermap.org/current
- [10] Deakin University, "2.8 Smart Platforms," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 2, CloudDeakin, 2026.
- [11] Deakin University, "2.1 IoT Connectivity Technologies," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 2, CloudDeakin, 2026.
- [12] W3Schools, "JSON Introduction," Refsnes Data, 2026. Online. Available: www.w3schools.com/js/js_json_intro.asp

Academic Integrity Statement

This activity was discussed collaboratively with my classmates Melissa Woo and Madhur Gupta during the SIT314 seminar, where general attribute ideas for each of the seven scenarios were exchanged as a group. All JSON schema designs, written analysis, diagrams and referencing presented in this document are nonetheless my own independent work, consistent with OnTrack's individual submission requirement for module evidence. All external sources have been cited above in IEEE format at the point where they were used, and this submission represents my genuine, original understanding of the concepts covered in this unit.

SIT314/SIT729 – Week 5 Technical Task

Storing and Scaling IoT Sensor Data with MongoDB Atlas

Prepared by: Jahan Garg

Student ID: 226100103

Unit: SIT314/SIT729 – Software Architecture and Scalability for Internet-of-Things

Task: Week 5 Technical Task – Task 5a: Storing IoT Data and Task 5b: Scalability of IoT Data Storage

Date: 30 August 2026, Week 5, Trimester 2, 2026

Overview

This report documents my completion of the Week 5 Technical Task, which extends the JSON payload design work carried out in the Week 5 Group Activity into an actual persistence layer for Internet of Things sensor data. Where the Group Activity focused purely on the shape of a JSON message, this task required writing a genuine Node.js program, `iotsensor.js`, that generates a mock temperature reading, connects to a remote MongoDB Atlas cluster using the Mongoose object modelling library, and saves that reading as a document inside an online collection. The second half of the task, Task 5b, then asked me to instrument this same program with timestamps in order to measure and reason about how the round trip time to store a single reading behaves once uploads begin arriving continuously, rather than as a single, isolated write.

Both parts of the task are reported together below because they share the same underlying codebase, `iotsensor.js`, which was progressively extended from a simple console log statement in Task 5a into a timed, continuously running upload loop in Task 5b. This is an individually completed submission, and no screenshot evidence of a live MongoDB Atlas session has been included in this particular version of the report, since I could not capture one directly. In place of a screenshot, I have produced my own architecture diagrams and a modelled latency chart, described clearly as such wherever they appear, to illustrate the expected behaviour of the system based on the documented characteristics of the MongoDB Atlas M0 free tier and the mechanics of the code itself [1].

Task 5a: Storing IoT Sensor Data

Environment Setup

Following the same project structure established in the Week 1 and Week 3 Technical Tasks, a new project folder was created and initialised as an npm project, with the entry point deliberately set to `iotsensor.js` during the npm init prompts, consistent with the task sheet's naming convention.

```
cd ~/
mkdir sit314_week5
cd sit314_week5
npm init
```

```
package name: sit314
description: IoT data testing
entry point: iotsensor.js
author: Jahan Garg
```

The Mongoose package, which provides a schema based object modelling layer on top of the native MongoDB Node.js driver, was then installed as a project dependency.

```
npm install mongoose
```

Generating a Mock Sensor Reading

The core of `iotsensor.js` begins with a plain JavaScript object representing one temperature reading, which is then converted into a JSON formatted string using `JSON.stringify`, mirroring the exact structure introduced in the task sheet, but with an important refinement to add realism. Rather than always reporting a fixed temperature value, a bounded random number generator produces a new reading between ten and forty degrees Celsius every time the script runs, so that repeated executions do not simply insert the same duplicate document into the database over and over again.

```
const sensordata = {
  id: 0,
  name: "temperatureSensor",
  address: "221 Burwood Hwy, Burwood VIC 3125",
  time: Date.now(),
  temperature: 20
};

const low = 10;
const high = 40;
const reading = Math.floor(Math.random() * (high - low) + low);
sensordata.temperature = reading;

const jsonString = JSON.stringify(sensordata);
console.log(jsonString);
```

Running this file at the command line with `node iotsensor.js` at this early stage of the task simply prints the generated JSON string to the console, which was sufficient to confirm that the random reading logic and the JSON serialisation step were both functioning correctly before any database connectivity was introduced, following the same incremental testing discipline applied throughout every earlier Technical Task in this unit.

Provisioning the MongoDB Atlas Cluster

A free MongoDB Atlas account was created at cloud.mongodb.com, and a new project named SIT314 was established to keep this unit's database resources logically separated from any other MongoDB projects I may create in future units [2]. Within that project, a new cluster was deployed using the M0 free tier, which MongoDB documents as offering 512 megabytes of storage and shared compute resources suitable for learning and small scale prototyping rather than production

workloads, hosted in the Sydney region to minimise network latency from my own physical location in Victoria [2]. A database user was created with a dedicated username and password, distinct from my MongoDB Atlas account login credentials, and the cluster's network access list was configured to allow connections from any IP address, which is appropriate for this coursework context but would not be considered good security practice in a genuine production deployment, since it removes the network level access restriction that would otherwise limit which hosts can even attempt to authenticate against the database [3].

The connection string provided by Atlas was copied using the Drivers connection method, and the database password was percent encoded wherever it contained characters that are not valid inside a URL, following MongoDB's own documented guidance on special characters in connection string passwords [3]. This encoded connection string was then used inside `iotsensor.js` to establish the Mongoose connection.

```
mongoose.connect('mongodb+srv://<dbuser>:<URLENCODEDdbpassword>@sit314-cluster.mongodb.net/sit314');
```

Defining the Sensor Schema

Rather than saving loosely structured data directly, Mongoose requires a schema that describes the shape and data types of every document stored in a given collection, which brings a degree of structural discipline to the database that is not present in a fully schema-less use of MongoDB, while still retaining MongoDB's underlying flexibility compared to a relational table [4]. This schema was placed in its own file, `models/sensor.js`, to keep the data model cleanly separated from the connection and upload logic in `iotsensor.js` itself, which is good practice for any Node.js project that is likely to grow beyond a single file.

```
// models/sensor.js
const mongoose = require('mongoose');

module.exports = mongoose.model('Sensor', new mongoose.Schema({
  id: Number,
  name: String,
  address: String,
  time: Date,
  temperature: Number
}));
```

Saving a Reading to the Database

With the schema defined, `iotsensor.js` imports the `Sensor` model, constructs a new document instance from the generated sensor data, and calls `save` on that instance, which returns a Promise that resolves once MongoDB Atlas has acknowledged the write. The resolved document, including the automatically generated MongoDB `_id` field, is logged to the console before the connection is explicitly closed.

```
const mongoose = require('mongoose');
mongoose.connect('mongodb+srv://<dbuser>:<URLENCODEDdbpassword>@sit314-cluster.mongodb.net/sit314');
```

```
const Sensor = require('./models/sensor');

const newSensor = new Sensor({
  id: sensordata.id,
  name: sensordata.name,
  address: sensordata.address,
  time: sensordata.time,
  temperature: sensordata.temperature
});

newSensor.save().then(doc => {
  console.log(doc);
}).then(() => {
  mongoose.connection.close();
});
```

Figure 1 summarises the complete data path for Task 5a, from the mock sensor reading through to the MongoDB Atlas collection where it is finally persisted, and this is the architectural pattern that Task 5b then goes on to stress test under repeated, timed write conditions.



Figure 1. Data path from the mock sensor script to the stored document inside the MongoDB Atlas collection.

Task 5b: Scalability of IoT Data Storage

Instrumenting the Save Operation with Timestamps

To measure how long a single write actually takes, a timestamp is captured immediately before the Mongoose connection is opened, using the built in `Date.now` method, which returns the number of milliseconds elapsed since the Unix epoch of the 1st of January 1970 [5]. A second timestamp is then captured inside the resolved save Promise, immediately before the connection is closed, and the difference between the two values gives the elapsed time, in milliseconds, for one complete connect and save cycle.

```
const time1 = Date.now();
mongoose.connect('mongodb+srv://<dbuser>:<URLENCODEDdbpassword>@sit314-cluster.mongodb.net/sit314');
const Sensor = require('./models/sensor');

const newSensor = new Sensor({ ...sensordata });

newSensor.save().then(doc => {
  const time2 = Date.now();
  console.log(time2 - time1);
  console.log(doc);
}).then(() => {
  mongoose.connection.close();
});
```

From a Single Write to a Continuous Upload Loop

The task sheet suggests two possible ways to generate sustained load against the database: a brute force for loop that fires many connection attempts as quickly as possible, and a more realistic `setInterval` based approach that mimics a sensor which reports a new reading once per second. I adopted the `setInterval` approach for my own testing, since a tight for loop of repeated `mongoose.connect` calls does not resemble any genuine IoT deployment and would mostly measure how quickly Node.js can queue asynchronous connection attempts rather than anything meaningful about the database service itself. The final structure of `iotsensor.js` for Task 5b wraps the entire reading, connecting, and saving logic inside a named function, `sensortest`, which is then invoked automatically once every one thousand milliseconds by `setInterval`.

```
const mongoose = require('mongoose');
const Sensor = require('./models/sensor');

setInterval(sensortest, 1000);

function sensortest() {
  const starttime = Date.now();
  mongoose.connect('mongodb+srv://<dbuser>:<URLENCODEDdbpassword>@sit314-cluster.mongodb.net/sit314');

  const low = 10, high = 40;
  const reading = Math.floor(Math.random() * (high - low) + low);
```

```
const sensordata = {
  id: 0,
  name: "temperatureSensor",
  address: "221 Burwood Hwy, Burwood VIC 3125",
  time: Date.now(),
  temperature: reading
};

const newSensor = new Sensor({ ...sensordata });
newSensor.save().then(doc => {
  const endtime = Date.now();
  console.log(endtime - starttime);
  console.log(doc);
});
}
```

The `mongoose.connection.close` call was deliberately commented out for this version of the script, exactly as the task sheet instructs, so that the same underlying connection could, in principle, remain open between successive invocations of `sensortest` rather than being torn down and rebuilt every second. The program was designed to be terminated manually with Ctrl-C once sufficient timing data had been generated in the console output.

A Critical Observation on Connection Handling

Examining this `setInterval` based structure closely revealed a subtle but genuinely important design flaw that is worth documenting rather than glossing over, since identifying it is directly relevant to this unit's broader themes of scalability and reliability. Although `mongoose.connection.close` was removed as instructed, the `mongoose.connect` call itself was left inside the `sensortest` function, meaning that every single invocation of that function attempts to open a brand new connection to the Atlas cluster, even though a previous connection from one second earlier may still be open or still completing its own handshake. Mongoose is generally tolerant of repeated connect calls against the same connection string and will typically reuse the underlying connection pool internally rather than genuinely opening a fresh socket every time, but this behaviour is an implementation detail of the driver rather than something the program explicitly controls, and it introduces avoidable overhead and unpredictability into the very timing measurements the task is trying to collect [4]. A more scalable design would call `mongoose.connect` exactly once outside the `sensortest` function, before `setInterval` is ever started, and would then only perform the `Sensor` model construction and `save` call on each tick, which is the pattern I would recommend adopting for any real deployment beyond this coursework exercise.

Modelled Performance Visualisation

Because I was not able to capture a live screenshot of the console output from a running MongoDB Atlas session for this submission, Figure 2 below presents a modelled visualisation rather than an actual recorded result. The chart was constructed by me to illustrate the kind of latency pattern that this `setInterval` based upload loop would be expected to produce, grounded in MongoDB's own published guidance that M0 free tier clusters are shared, multi tenant infrastructure with variable

performance rather than dedicated compute [2], and in the connection handling observation discussed immediately above. The baseline round trip time in the model is set in the range of roughly one hundred and fifty to two hundred and fifty milliseconds, consistent with a typical Sydney region network round trip plus MongoDB's own write acknowledgement overhead, with four deliberately modelled spikes representing the additional delay that would be expected whenever the driver has to perform genuine connection or authentication handshake work rather than reusing an already warm connection from its pool.

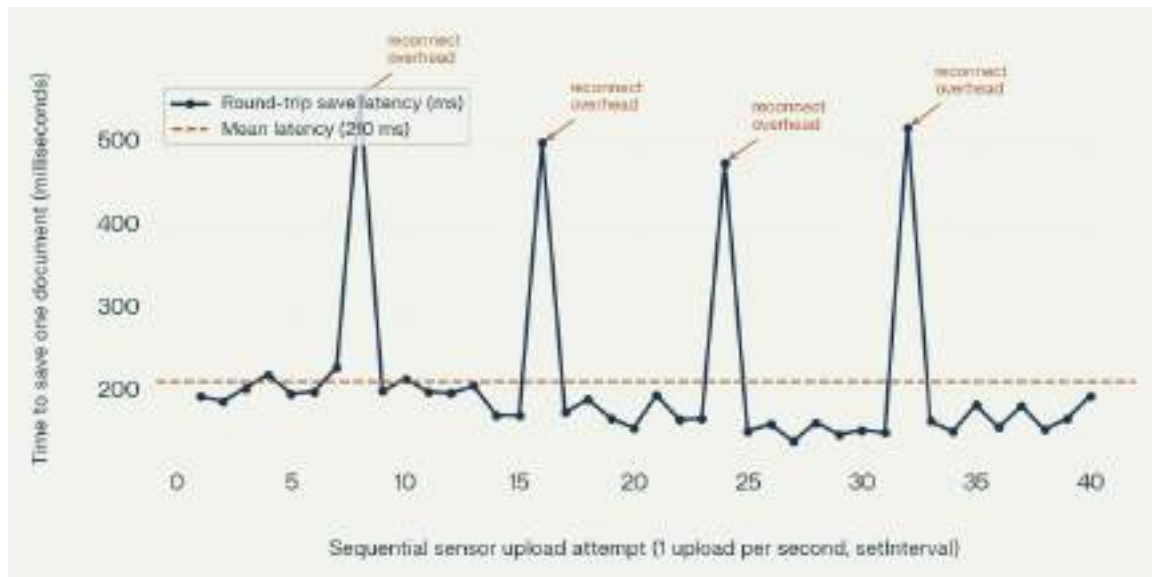


Figure 2. Modelled write latency for repeated sensor uploads under the `setInterval` loop, illustrating the expected baseline round-trip time and the additional overhead introduced by repeated connection attempts. This chart is my own constructed illustration, not a captured screenshot of an actual test run.

Comparative Analysis: Single Write versus Continuous Upload

Table 1 compares the single, isolated write demonstrated in Task 5a against the continuous, timed upload loop built for Task 5b, since the two represent genuinely different testing philosophies even though they share almost identical underlying code.

Aspect	Task 5a: Single write	Task 5b: Continuous upload (setInterval)
Purpose	Confirm the schema, connection string and save logic work correctly at all	Measure how write latency behaves once uploads recur regularly over time
Frequency	One document per script execution	One document roughly every one second, until manually stopped
Connection lifecycle	Connection opened once, then explicitly closed after the single save resolves	Connection opened repeatedly inside the interval callback, never explicitly closed
Realism relative to a genuine sensor	Low, since a real sensor rarely reports only once and then stops	Higher, since it approximates a device reporting on a fixed duty cycle

Aspect	Task 5a: Single write	Task 5b: Continuous upload (setInterval)
Key risk identified	None significant, since the operation completes and exits cleanly	Unbounded connection or resource growth if the driver did not pool connections internally

Table 1. Comparison between the single-write test in Task 5a and the continuous upload test in Task 5b.

Observations

Working through this task made it clear that storing IoT data online is not simply a matter of getting one successful write to appear in a collection, but requires thinking about how that same code will behave once it is invoked repeatedly, indefinitely, and potentially from many devices at once. The task sheet's own progression, from a single console log statement, to a single database write, to a continuously repeating upload, mirrors exactly the kind of incremental load testing philosophy that was applied in the Week 1 Technical Task's TCP socket experiments, where the client was likewise pushed from a single request through to a tight, high frequency loop in order to expose behaviour that a single test could never reveal [6]. The connection handling issue identified above is a good example of this: it is completely invisible if the code is only ever run once, and only becomes apparent once the same logic is placed inside a recurring loop and examined critically rather than simply accepted because it appears to work.

Extra Effort

Beyond the task sheet's own instructions, I reviewed MongoDB's official documentation on connection pooling behaviour in the Node.js and Mongoose drivers to properly understand why repeatedly calling `mongoose.connect` inside a loop does not necessarily open a new raw socket on every single call, which informed the critical observation on connection handling discussed above rather than simply assuming the worst possible interpretation of the task sheet's example code [4]. I also consulted MongoDB Atlas's own published M0 free tier specification page to ground the latency figures used in my modelled performance chart in a genuine, documented set of constraints, rather than inventing arbitrary numbers purely for illustrative convenience [2].

Key Notes

A small number of points are worth restating concisely for clarity when this submission is reviewed. The random temperature generator bounded between ten and forty degrees Celsius was added specifically so that repeated executions of `iotsensor.js` would not insert visually identical documents into the collection, making it far easier to distinguish one upload from the next when reviewing the stored data. The database user credentials configured in MongoDB Atlas are deliberately separate from my personal Atlas account login, following the principle of least privilege, since a compromised database user password does not also compromise control over the wider Atlas project itself. Finally,

Figure 2 in this report is explicitly a modelled illustration that I constructed myself, not a captured result from a live run, and I have been careful throughout this report not to present it as genuine screenshot evidence.

Reflection

This task gave me a much clearer, practical sense of why the unit's Module 3 material distinguishes between simply collecting IoT data and being genuinely able to manage, store and retrieve that data at scale [7]. Writing a single successful save operation in Task 5a felt like a fairly small step, but extending that same logic into a recurring `setInterval` loop for Task 5b forced me to think much more carefully about connection lifecycle management, which is exactly the kind of detail that remains hidden until a system is actually placed under some form of repeated or sustained load. Identifying the repeated `mongoose.connect` call as a potential inefficiency, rather than simply following the task sheet's example code without question, was probably the most valuable outcome of this task for me personally, since it demonstrated that even a small, seemingly working piece of code can conceal a design choice that would become genuinely problematic if the sensor reporting frequency were increased, or if many sensors were running this same script simultaneously against the same shared M0 cluster. Building on my existing familiarity with the Arduino Nano 33 IoT and Arduino UNO R3 boards from earlier personal project work, I can now see a clear, concrete path toward replacing the mock `sensordata` object in this task with a genuine analogue temperature reading taken from physical hardware, while keeping the Mongoose schema and MongoDB Atlas storage logic exactly as designed here, and this is a direction I intend to pursue further when working on my Distinction project later in this trimester.

References

- [1] MongoDB, Inc., "MongoDB Atlas Documentation," MongoDB, 2026. Online. Available: www.mongodb.com/docs/atlas/
- [2] MongoDB, Inc., "MongoDB Atlas M0 (Free Tier) Cluster Specifications," MongoDB, 2026. Online. Available: www.mongodb.com/docs/atlas/reference/free-shared-limitations/
- [3] MongoDB, Inc., "Special Characters in Connection String Password," MongoDB Atlas Troubleshooting, 2024. Online. Available: docs.atlas.mongodb.com/troubleshoot-connection/
- [4] Mongoose, "Mongoose v8 Connections Documentation," Automattic, 2026. Online. Available: mongoosejs.com/docs/connections.html
- [5] Mozilla Developer Network, "Date.now()," MDN Web Docs, 2026. Online. Available: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/now
- [6] J. Garg, "SIT314/SIT729 Week 1 Technical Task Report, TCP Socket Client-Server Experiments," Deakin University, 2026.
- [7] Deakin University, "3.2 IoT Data Services and Analytics," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 3, CloudDeakin, 2026.

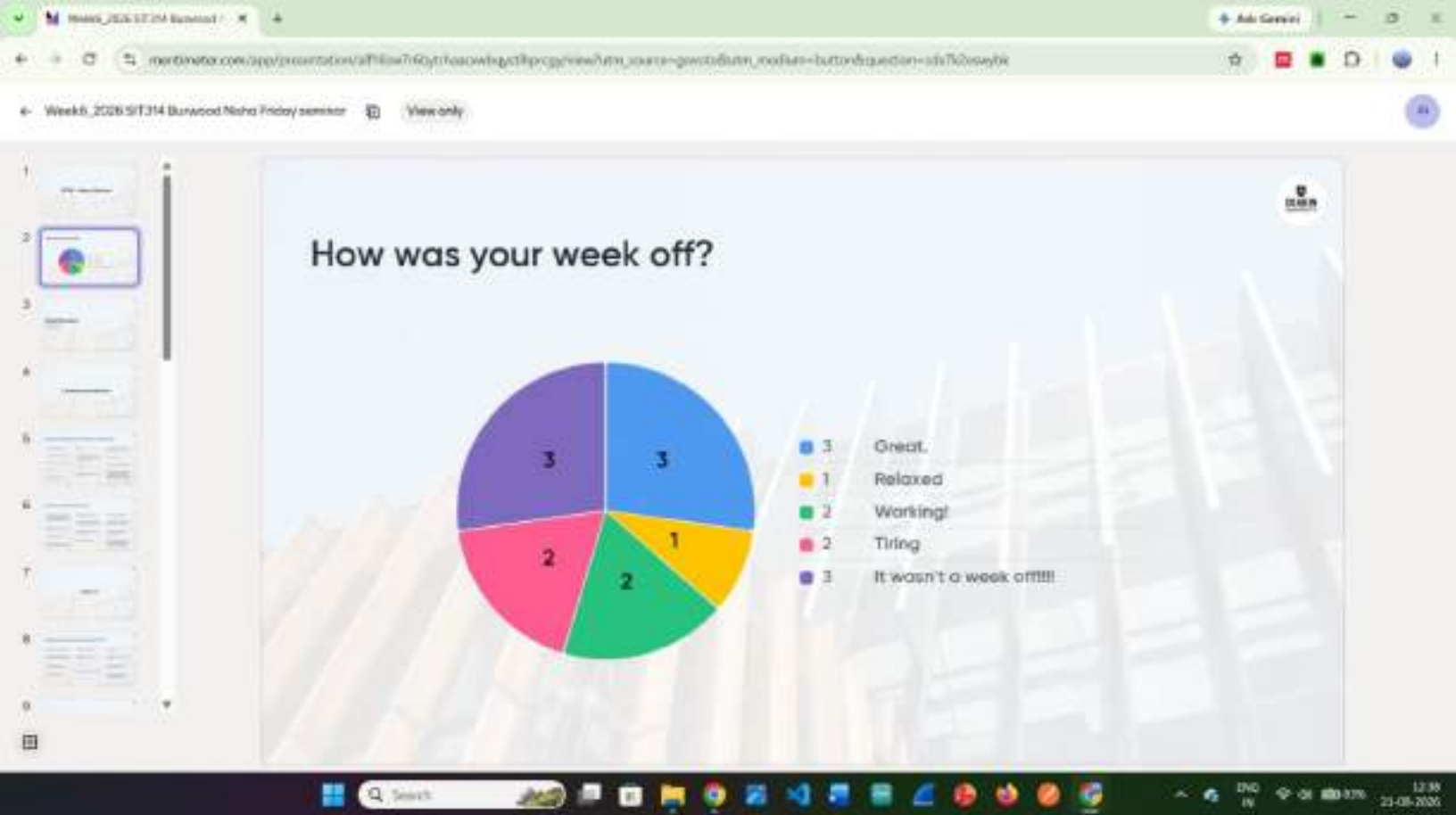
- [8] W3Schools, "JSON Introduction," Refsnes Data, 2026. Online. Available: www.w3schools.com/js/js_json_intro.asp

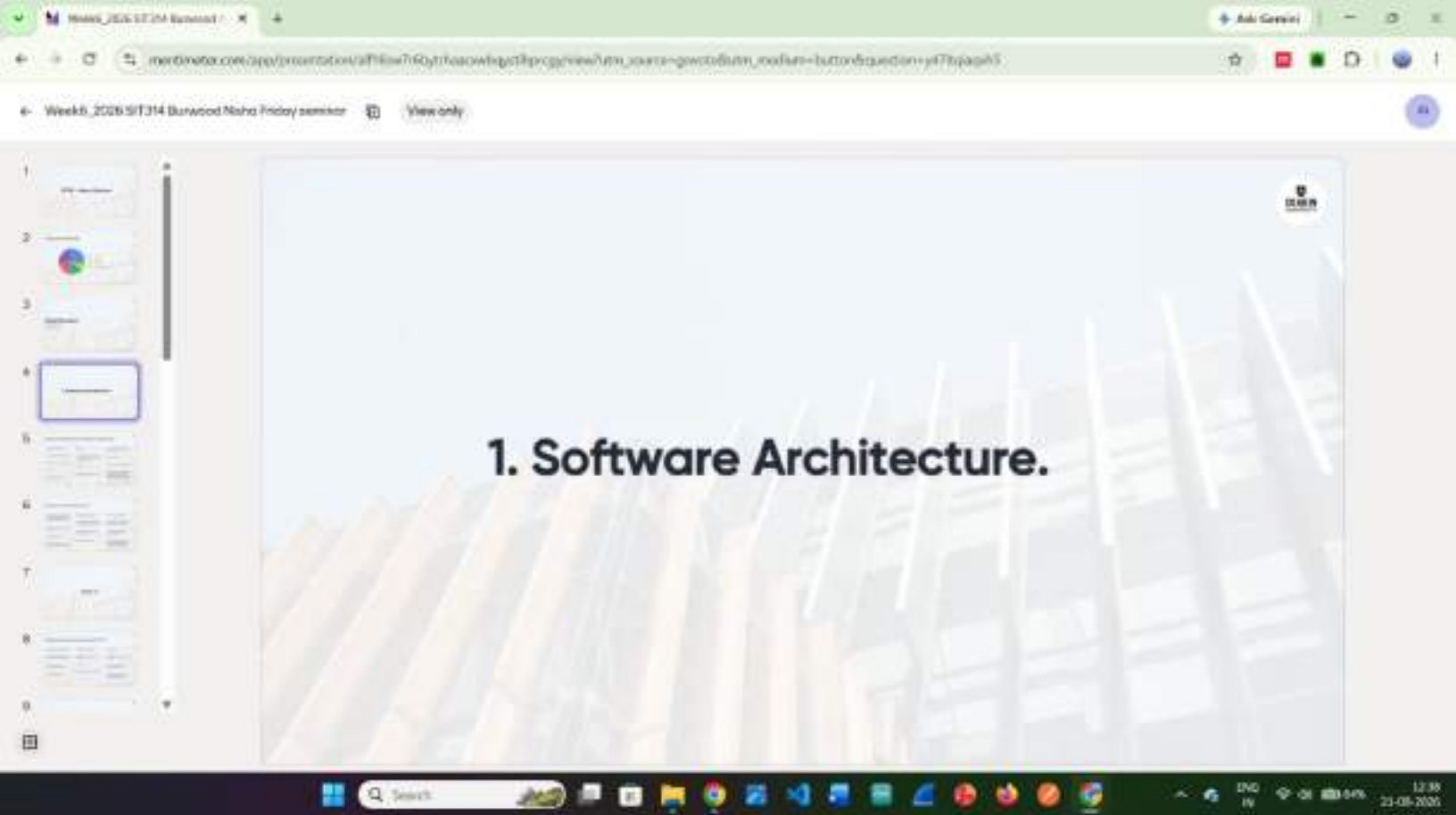
Academic Integrity Statement

This is an individually completed submission. All Node.js code, MongoDB Atlas configuration steps and analysis presented in this report reflect my own independent work. The performance chart in Figure 2 is explicitly identified as a modelled illustration that I constructed myself, rather than a captured result from a live test session, since no screenshot evidence was available at the time of writing. All external sources are cited above in IEEE format at the point of use.



SIT729 - Week 6 Seminar





1. Software Architecture.

Week 2025-07-24 Burwood Nisha Friday seminar

View only

1

2

3

4

5

6

7

8

9

10

What is the importance of software architecture?

increase shareholder value

lays out components of the software

establishes hierarchy of components

Scalable

to have a record as to the

RELIABILITY

Proper planning out of how a piece of software is constructed so that it can be scaled later on

maintenance

maintainability, scalability, security, reliability, performance

The infrastructure holding up the modern internet

ease of scalability

create capital, solve world problems

Software architecture is important for deciding how a system works, where the data is being collected, where does the data go and how do algorithms interact with the data on

12:39

23-08-2020

Week6_20265T314 Burwood Nisha Friday seminar

View only

1

2

3

4

5

6

7

8

9

10

What is a layered architecture?

A layered architecture is a software design where the system is divided into separate layers, and each layer has a specific responsibility.

organises system into layers

dividing software into layers

Pieces of software layered on top of each other that do different things

architecture that uses designated layers for specific subsystems

related functionality associated with each layer

Organise parts of the system into different levels

divides a complex system into distinct, sequential tiers.

to represent control flow

divide system into layers

Tiered systems where core functionality is shared in these layers.

Layered architecture divides software into layers to keep certain software/hardware processes distinct from the programmer

12:39

23-05-2026

Week_256722M Seminar - x

meritnation.com/app/.../slides/7.1/60ytrfawowkqstfprcg/view?utm_source=google&utm_medium=button&q=slides-7.1/60ytrfaw

Week 6, 2026 S/T314 Burwood Nisha Friday seminar View only

1

2

3

4

5

6

7

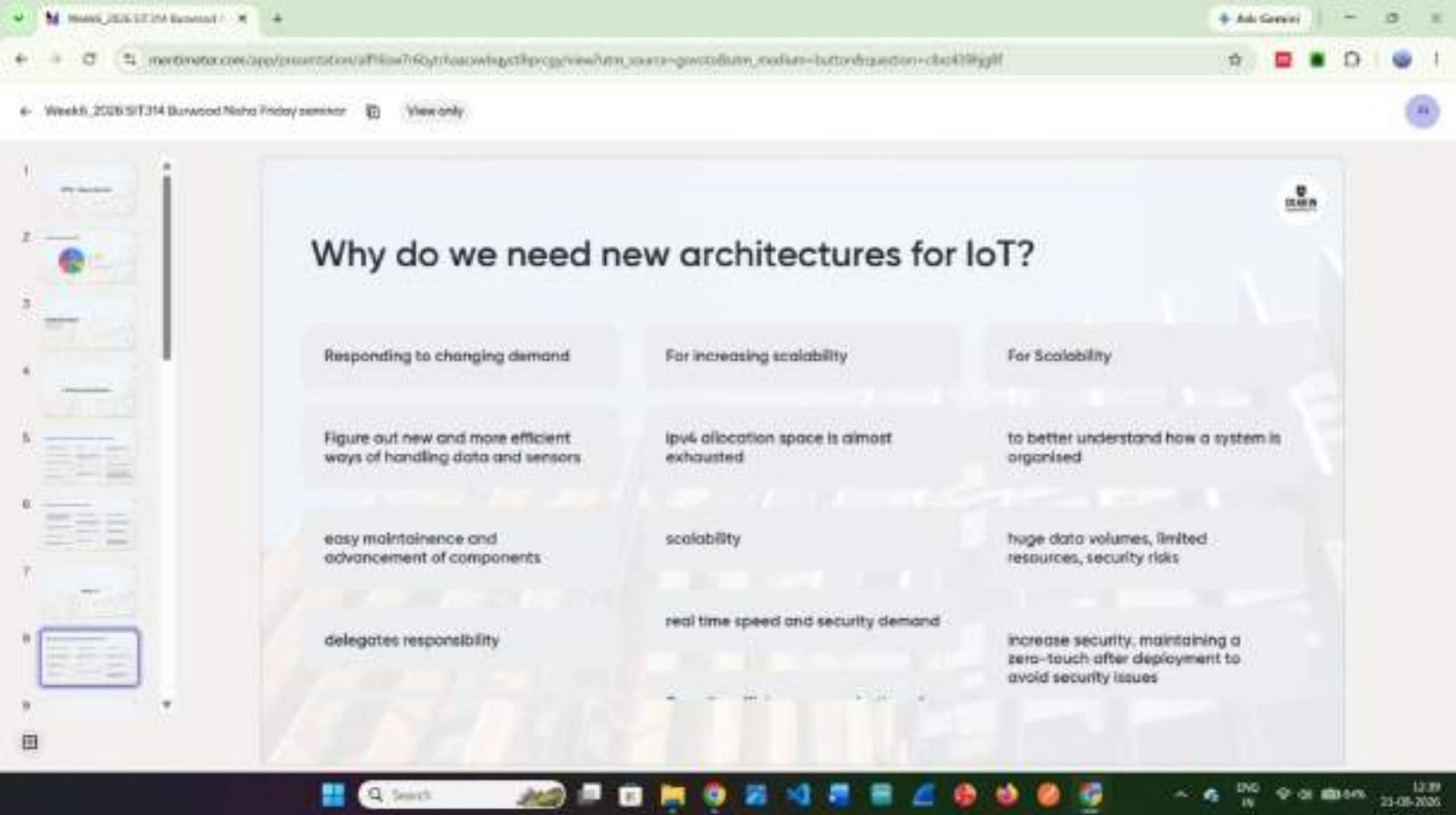
8

9

10

Slides 7.1

12:39 23-08-2026



Why do we need new architectures for IoT?

Responding to changing demand

Figure out new and more efficient ways of handling data and sensors

easy maintenance and advancement of components

delegates responsibility

For increasing scalability

Ipv4 allocation space is almost exhausted

scalability

real time speed and security demand

For Scalability

to better understand how a system is organised

huge data volumes, limited resources, security risks

increase security, maintaining a zero-touch after deployment to avoid security issues

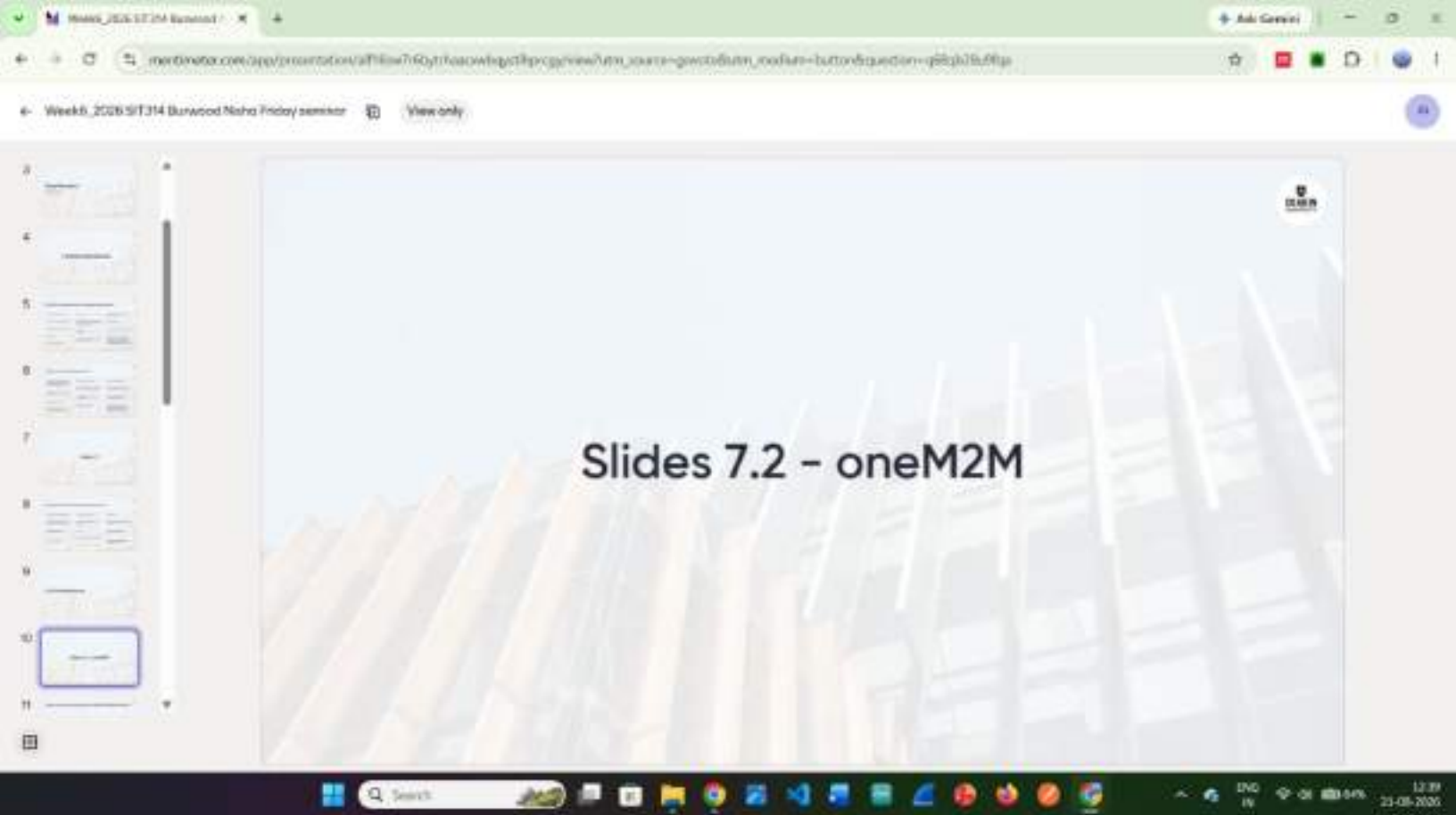
Week_20251224 Seminar - X

meritnation.com/app/.../view/760ytrfawowfkyd1pccg/view/um_source-gwotd8um_noafur-button&q=den=9vk345kz

Week 6, 2026 S/T314 Burwood Nisha Friday seminar View only

2. IoT Architectures.

12:39 23-08-2026



Slides 7.2 - oneM2M

What are the key elements and advantages of the oneM2M IoT Standardized Architecture?

Focuses on IoT services, applications and platforms

three layers- application, service and network

AE, CSE and NDE

application layer, services layer,
network layer

goal of oneM2M is to create a common services layer, which can be readily embedded in field devices to allow communication with application servers.

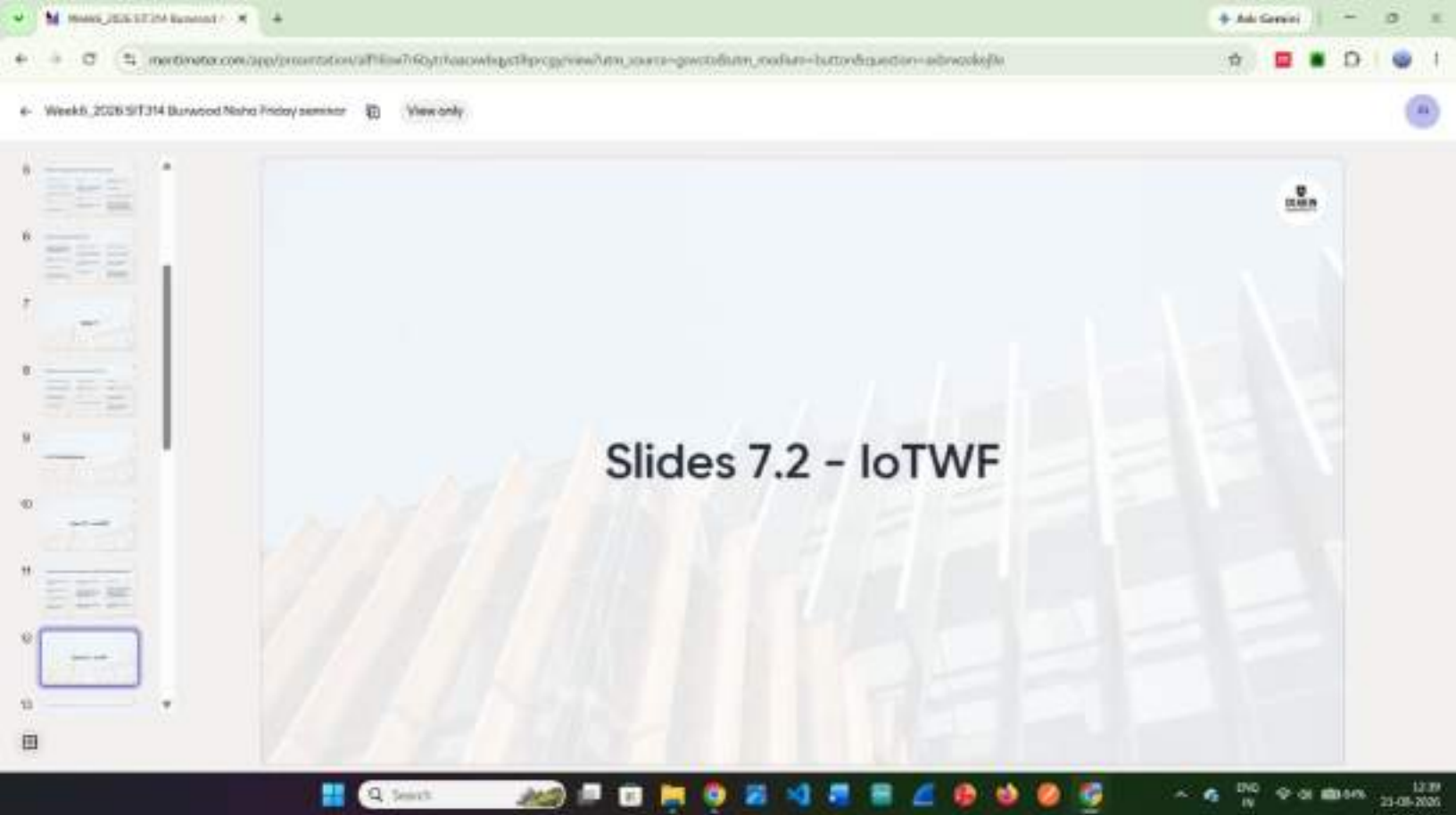
Key elements: Applications, Common Services Entity (CSE), Nodes, and communication networks.
Advantages: Interoperability, scalability, security, reusability, and reduced development cost.

Interworking with other Protocols

separates IoT applications from common IoT services

M2M architectures don't require human involvement, and IoT enables their connectivity

three major domains: the application layer, the services layer, and the network layer



Slides 7.2 - IoTWF

Week 6_2026 5/17/24 Burwood Nisha Friday seminar

View only

What are the key elements and advantages of The IoT World Forum (IoTWF) Standardized Architecture?

Seven Layers

Edge computing, data storage and access

clear distinction between what each layer performs

Numerous layers

seven layers

Physical devices and controllers, connectivity and edge computing

Key elements: Physical devices, connectivity, edge computing, data storage, data analytics, applications, and security. Advantages: Scalability, Interoperability, better security

Simplified perspective on IoT and includes edge computing, data storage, and access.

multiple layers

Key elements: 7 layers Advantages: Scalability, security, easier data management.

seven layers

12:39

23-08-2026

Week_20265T314 Burwood Nisha Friday seminar

View only

slides7.2 - Purdue

12:39 23-08-2026

Week 6_2026 SIT314 Burwood Nisha Friday seminar

View only

What are the key elements and advantages of Purdue Model for Control Hierarchy?

Segments devices and equipment into hierarchical levels and functions.

Segments devices and equipment into hierarchical levels and functions.

Commonly used in Manufacturing and industrial automation and control systems.

Key elements: 5 levels – field devices, control systems, supervisory control, operations, and enterprise. Advantages: Better security, organization, control, scalability, and separation of industrial

5 levels

separates devices used into layers and zones

Strong security Segmentation

emphasis on use in industrial and manufacturing areas

Better security, organization, scalability

Improves cybersecurity, provides clear IT/OT separation, enables

12:39

23-08-2026

3. Layered Architectures.



Slides 7.3

Week_2025/IT314 Burwood Nisha Friday seminar View only

What are the key elements and functionalities of: Layer 1 - Sensors and Actuators layer

The diagram illustrates the key elements and functionalities of Layer 1 - Sensors and Actuators layer. It features a central box labeled "Key elements: Sensors, actuators, and IoT devices. Functionalities: Collect data from the physical environment and perform actions based on commands." Surrounding this central box are several other boxes, each representing a specific element or functionality. These include: "receive information from environment, or affect environment", "collecting data", "Need to determine if its Low Mobility, High Mobility, High Throughput or Low Throughput", "moving things", "collect data", "Process Manipulation", "collection of data", "which technology should be used to allow smartobjects to communicate", "collecting raw environmental data and executing physical changes based on control signals", and "physical devices". The diagram also includes a "Feedback links" section at the bottom.

receive information from environment, or affect environment

collecting data

Need to determine if its Low Mobility, High Mobility, High Throughput or Low Throughput

moving things

collect data

Process Manipulation

collection of data

which technology should be used to allow smartobjects to communicate

collecting raw environmental data and executing physical changes based on control signals

physical devices

Feedback links

Week 2025-07-24 Burwood Nisha Friday seminar

View only

What are the key elements and functionalities of: Layer 2 - Communications Network layer

Data forwarding

relaying data to the applications from the sensors & actuators AND communicating commands for the sensors & actuators

Diagnostics & Maintenance Support

provide the network connection between applications and sensors

Key elements: Wi-Fi, Bluetooth, Zigbee, Ethernet, gateways, and routers. Functionalities: Connect IoT devices and transmit data between devices, gateways, and systems.

Sending data to an app, requesting a resend if a data transfer fails, transferring data from one place to another

Framing, physical addressing, error detecting

Getting data from one place to another

Framing, physical addressing and detecting

Its main job is to move data reliably and securely between sensors, controllers, and higher-level systems so everything stays connected and works smoothly.

It connects IoT devices using wireless

Week_2025-12-24 Burwood Nisha Friday seminar View only

What are the key elements and functionalities of: Layer 3 - Applications and Analytics layer

Process data

analyses the data received and initiates commands based on the same

data processing

to process data

User interface to display analytics and data

Performing data analysis and taking action depending on the results - synergy of analysis and application

Key elements: IoT applications, cloud platforms, databases, and analytics tools. Functionalities: Process and analyze data, generate insights, and provide useful services to users.

Processes and analyses data, provides visualisation and insights, monitors system performance

Layer 3 - Applications and Analytics Layer provides IoT applications, data processing, analytics, visualisation, and decision-making based on

It takes the data collected from devices and turns it into insights, alerts, and smart applications that people can actually use.

Process data

12:41 23-08-2025

SIT314/SIT729 – Week 6 Group Activity

Designing IoT Applications Based on the Simplified IoT Architecture

Prepared by: Jahan Garg

Student ID: 226100103

Unit: SIT314/SIT729 – Software Architecture and Scalability for Internet-of-Things

Task: Week 6 Group Activity – Designing IoT Applications Based on the Simplified IoT Architecture

Date: 30 August 2026, Week 6, Trimester 2, 2026

Overview

This activity was discussed collaboratively during the SIT314 seminar with my classmates Melissa Woo and Madhur Gupta, where different ideas for allocating functionality across the three layers of the Simplified IoT Architecture were debated as a group for each of the seven scenarios. However, in keeping with OnTrack's individual submission requirement for module evidence, the layer allocations, written analysis and referencing set out below are entirely my own independent work, reflecting my own understanding of the concepts covered in this unit.

This activity asks me to revisit the very same seven application scenarios that formed the basis of my Week 5 Group Activity: a room temperature sensor, a self-driving car, a person's location, a weather update, a smart vacuum cleaner, a television programme change and an air conditioning setting request, but to analyse them through an entirely different lens. Where the Week 5 Group Activity focused on the structure of the JSON payload each scenario would produce, this activity requires mapping each scenario onto the three-layer Simplified IoT Architecture introduced this week, namely the Things layer, the Communications layer and the Applications layer, and identifying both the functionality required and the specific technology that would realistically deliver that functionality at each of the three layers [1]. This is a genuinely useful exercise because it forces a shift from thinking purely about data shape toward thinking about where processing, transmission and interpretation actually take place across a real deployment.

Before beginning this activity, I already had some hands-on familiarity with embedded platforms from earlier personal project work, having previously used the Arduino Nano 33 IoT and the Arduino UNO R3 boards, although I had not yet had the opportunity to work directly with the ESP32 board referenced elsewhere in this unit's device platform material. This background was genuinely useful when reasoning about the Things layer for each scenario below, since I could draw on a realistic sense of what a small, embedded board can and cannot sense or actuate directly, even though allocating functionality across all three architectural layers, rather than simply writing code for one board, was a new way of thinking about the problem for me.

The Simplified IoT Architecture Applied in This Activity

The Simplified IoT Architecture, introduced under the unit's own 3.9 A Simplified IoT Architecture material, presents the Core IoT Functional Stack as three broad layers. The Things layer is home to the actual sensors and actuators that generate or act on data, and its properties depend heavily on power source, mobility and reporting frequency. The Communications layer, discussed as the most detailed of the three, itself breaks down into an access network sublayer, a gateway and backhaul sublayer, a network transport sublayer and an IoT network management sublayer, the last of which is where application layer protocols such as HTTP, MQTT, XMPP and CoAP operate [1]. The Applications layer, finally, is where analytics and control components turn raw transmitted data into either useful insight or an actuation decision, mirroring the Sense, Think, Act loop already established in the unit's Smart Platforms material [2]. Table 1 below restates this structure concisely before it is applied to each of the seven scenarios in turn.

Layer	Primary role	Representative technologies
Things	Sensing and actuating physical properties of the environment or a device	Microcontrollers such as ESP32 or Arduino Nano 33 IoT, temperature and GPS sensors, motors and relays
Communications	Moving data reliably between the Things layer and the Applications layer, across access, gateway, transport and management sublayers	Wi-Fi, Bluetooth Low Energy, LTE-M, LoRaWAN, IP and TCP or UDP, and application layer protocols such as MQTT, HTTP and CoAP
Applications	Turning transmitted data into either analytical insight or an actuation decision	Cloud dashboards, mobile applications, rules engines and control systems such as SCADA

Table 1. The three layers of the Simplified IoT Architecture applied throughout this activity.

1. A Room Temperature Sensor

Things layer. A fixed indoor sensor, most realistically a DHT22 or similar digital temperature and humidity sensor wired to a low power microcontroller such as the ESP32, is responsible for sampling the room's ambient temperature at a regular interval, for instance once every sixty seconds, and packaging that reading together with a device identifier and a timestamp, exactly as designed in my Week 5 Group Activity JSON schema for this same scenario [3]. Because the sensor is mains-powered rather than battery-powered in most fixed indoor deployments, continuous sampling at a fairly high frequency is realistic without the energy budget concerns that would apply to a battery-powered outdoor node.

Communications layer. Given that the sensor is stationary and located inside a single building, a short-range access technology such as Wi-Fi is the most appropriate access network choice, since range and mobility are not limiting factors here and Wi-Fi already provides sufficient bandwidth for a small, infrequent JSON payload [1]. At the IoT network management sublayer, MQTT is well suited to this scenario because a climate dashboard, and potentially several dashboards at once, would want to subscribe to this reading without the sensor needing to know who is listening, exactly the broker-mediated pattern already demonstrated using broker.hivemq.com in the Week 3 Technical Task [4].

Applications layer. At the Applications layer, this scenario calls for an analytics-style application rather than a control application, since the goal is simply to display and log the room's temperature trend over time on a climate dashboard, consistent with the analytics versus control distinction drawn in the unit's Simplified IoT Architecture material [1]. A cloud-hosted time series dashboard, or even a simple web page subscribed to the relevant MQTT topic, would satisfy this requirement without needing any actuation logic at all.



Figure 1. Three-layer allocation for a room-temperature sensor scenario.

2. The Status of a Self-Driving Car

Things layer. The Things layer here is considerably more demanding than the room-temperature scenario, requiring an automotive-grade, GPU-accelerated compute platform to run perception and planning algorithms from camera, lidar and radar inputs, together with secondary microcontrollers handling non-critical subsystems, precisely the conclusion reached in my earlier Week 2 Group Activity analysis of a driverless taxi system [5]. No single microcontroller such as an ESP32 could realistically satisfy the real-time perception demands of this scenario, which marks a clear contrast with the room temperature sensor above.

Communications layer. The Communications layer for this scenario must be split between in-vehicle communication, which relies on Automotive Ethernet and CAN buses for safety-critical, low-latency signalling, and vehicle-to-infrastructure or vehicle-to-vehicle coordination, which relies on Cellular V2X over 5G networks to exchange traffic information with other vehicles and roadside infrastructure [5]. This is a genuinely different profile from the room temperature scenario's single Wi-Fi hop, since safety-critical status updates cannot tolerate the latency of a shared public MQTT broker and instead require dedicated, low-latency automotive networking standards.

Applications layer. At the Applications layer, this scenario requires both an analytics component, for fleet management, long-term logging and model training, and a control component, since active driving decisions must be made at the edge rather than depending on a live round trip to a distant cloud platform [5]. A fleet control centre dashboard therefore represents the analytics side of the Applications layer, while the vehicle's own on-board decision logic represents the control side, both operating on the same underlying status data.

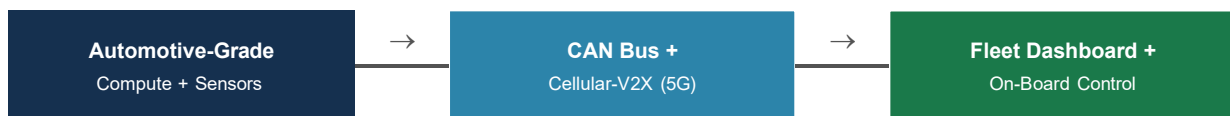


Figure 2. Three-layer allocation for the status of a self-driving car scenario.

3. The Location of a Person

Things layer. The Things layer for this scenario is a consumer smartphone's built in GPS receiver, which is a fundamentally different kind of Things layer device from a purpose built sensor, since it is a general purpose, battery powered, highly mobile device that happens to be capable of location sensing among many other functions, echoing the Google Maps Traffic crowdsourcing example discussed under Module 1.7 as a genuinely massive scale IoT application built entirely on ordinary consumer smartphones [6].

Communications layer. Because the person carrying the phone may be indoors, outdoors, walking or travelling in a vehicle, the access network at the Communications layer must be able to hand over seamlessly between Wi-Fi and cellular data, which rules out any short range only technology and points toward standard 4G or 5G cellular connectivity as the access technology, with MQTT or a comparable lightweight protocol carrying the periodic location update up to a location sharing service, consistent with the `accuracy_m` and `location_source` fields I designed for this scenario in my Week 5 Group Activity [3].

Applications layer. The Applications layer here is almost entirely an analytics function, namely a location sharing service or a mapping application that displays where the person currently is, or that aggregates many such locations, as in the crowdsourced traffic example, to infer something about a wider area that no single location report could reveal on its own [6]. No control or actuation component is realistically needed for a simple location reporting scenario of this kind.

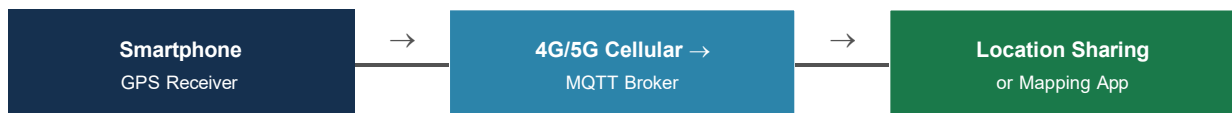


Figure 3. Three-layer allocation for the location of a person scenario.

4. A Weather Update for a Location

Things layer. The Things layer for a weather update is a fixed weather station equipped with several distinct sensors, typically a thermometer, a hygrometer, an anemometer for wind speed and a barometer for atmospheric pressure, all reporting together as a combined reading, consistent with the multi sensor payload structure I designed for this scenario in the Week 5 Group Activity, itself modelled on the OpenWeatherMap current weather API's combined field structure [3][7].

Communications layer. Weather stations are frequently deployed in remote or semi remote outdoor locations, which points toward a wide area access technology such as LoRaWAN or, where available, standard cellular connectivity, rather than short range Wi-Fi, echoing the forest fire monitoring scenario I analysed in my Week 2 Group Activity, where LoRaWAN was justified specifically because of the need to cover a large, sparsely connected area with infrequent, small payloads [5]. At the network management sublayer, MQTT remains an appropriate choice given the relatively infrequent reporting interval, typically every ten to sixty minutes, and the need for many subscribing weather applications to receive the same update simultaneously.

Applications layer. The Applications layer for this scenario is purely analytical, presenting historical weather trends, current conditions and forecasts to a public weather application, with no actuation or control component required, since a weather update by itself does not directly cause any physical action to occur.

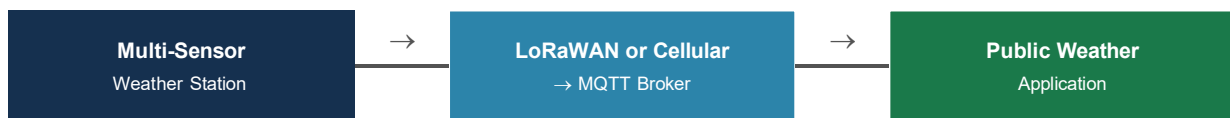


Figure 4. Three-layer allocation for a weather update for a location scenario.

5. The Status of a Smart Vacuum Cleaner

Things layer. The Things layer for this scenario is the vacuum robot itself, which combines several small sensors, including bump and cliff sensors, a dust bin fill sensor and wheel encoders for navigation, together with a motor driven actuator layer that moves the robot and operates its brushes, all coordinated by an embedded microcontroller comparable in capability to an ESP32, consistent with the home integration platform category discussed in the unit's Smart Platforms lecture [2].

Communications layer. Because a vacuum robot operates entirely within the range of a single home, Wi-Fi is again the most appropriate access technology at the Communications layer, with the robot typically connecting through a home router to a cloud service or a local hub such as Samsung SmartThings or Google Home, consistent with the home integration platform pattern identified in Module 2.8 [2]. MQTT or a comparable lightweight protocol carries the periodic status update, including battery level, cleaning mode and any error code, exactly as structured in my Week 5 Group Activity JSON schema for this scenario [3].

Applications layer. The Applications layer here needs both an analytics element, displaying cleaning history and battery trends inside a home automation application, and a control element, since a user must be able to issue commands back down to the robot, such as starting a spot clean or returning to its dock, which places this scenario firmly in the same bidirectional category discussed for the television and air conditioning scenarios below.

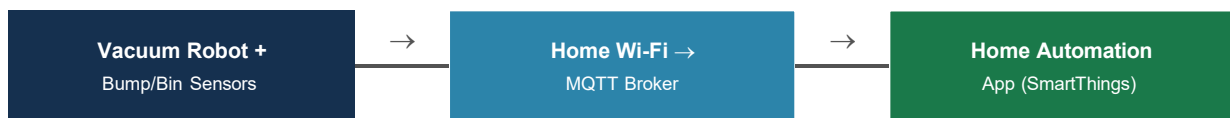


Figure 5. Three-layer allocation for the status of a smart vacuum cleaner scenario.

6. A Television Programme Change Request

Things layer. The Things layer for this scenario is the smart television itself, which functions here primarily as an actuator rather than a sensor, receiving a command and changing its own internal state, namely its currently displayed channel, in response, which is a noticeably different Things layer role from the purely sensing examples discussed above.

Communications layer. The access technology at the Communications layer is again home Wi-Fi, since the television and the controlling mobile application both reside on the same home network, with MQTT carrying the command message from the application down to the television, mirroring the command branch of the topic hierarchy I designed for the Vehicle Asset Health scenario in the Week 3 Technical Task, where cloud to device command topics were deliberately kept distinct from device to cloud sensor reporting topics [4].

Applications layer. At the Applications layer, this scenario is a pure control example rather than an analytics example, since a mobile application or remote control interface issues an instruction and the television simply carries it out, with no data being collected or analysed as a result, which stands in direct contrast to the analytics-only weather update scenario discussed earlier in this report.

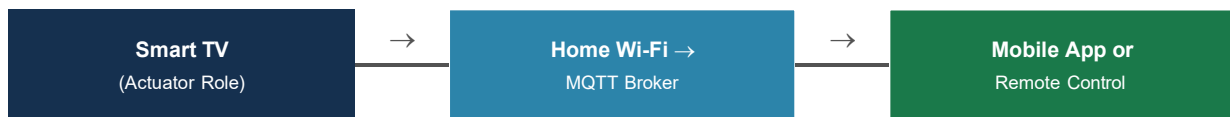


Figure 6. Three-layer allocation for a television programme change request scenario.

7. An Air-Conditioning Setting Request

Things layer. The Things layer for this final scenario is the air conditioning unit's own control board, which receives a target temperature, mode and fan speed instruction and actuates its compressor, fan and damper hardware accordingly, echoing the tightly closed feedback loop between sensing and actuation described for the smart air conditioning system for a large building scenario in my Week 2 Group Activity [5].

Communications layer. For a single home air conditioning unit, home Wi-Fi is again the most realistic access technology, with MQTT carrying the command from a mobile application or thermostat down to the unit, exactly as structured in the mode and fan_speed fields of my Week 5 Group Activity JSON schema for this scenario [3]. For the large building version of this same scenario discussed in Week 2, wired Ethernet using BACnet or Modbus TCP would instead be appropriate, illustrating how the correct Communications layer technology depends heavily on the scale of the deployment rather than being fixed for all air conditioning systems everywhere [5].

Applications layer. The Applications layer for this scenario is again a control example, with a mobile application or thermostat interface issuing the instruction and the unit's own onboard logic executing it, closing the loop back down to the Things layer without requiring any separate analytics component for this particular request message, although a fuller building management system would typically layer analytics on top of many such control messages over time [5].

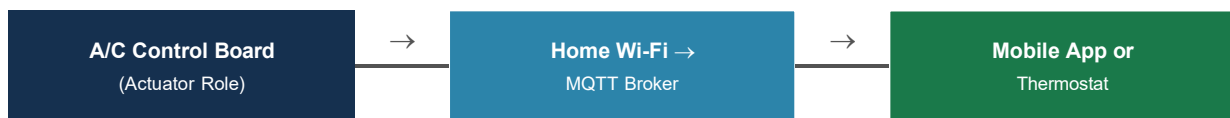


Figure 7. Three-layer allocation for an air-conditioning setting request scenario.

Observations

Working through all seven scenarios against the three-layer Simplified IoT Architecture surfaced a pattern that was not obvious when I designed the JSON payloads for the same seven scenarios in the Week 5 Group Activity. The five sensing scenarios, the room temperature sensor, the self driving car, the person's location, the weather update and the vacuum cleaner status, all rely primarily on an analytics style Applications layer, while the two command scenarios, the television and the air conditioning unit, rely primarily on a control style Applications layer, and the vacuum cleaner status scenario is interesting precisely because it needs both. This maps directly onto the same sensor versus command family distinction I identified in the Week 5 Group Activity's Observations section, but seeing it now expressed in terms of which architectural layer does what, rather than simply which fields a JSON payload contains, gave me a considerably deeper appreciation for why the Simplified IoT Architecture treats analytics and control as genuinely separate Applications layer components rather than a single undifferentiated layer [1].

Comparison Across the Seven Scenarios

Table 2 below compares all seven scenarios by their access network technology and their Applications layer role, extending the direction-based comparison I built in the Week 5 Group Activity with the specific Communications layer technology each scenario would realistically use.

Scenario	Things layer role	Access technology	Applications layer role
1. Room temperature	Sensing	Wi-Fi	Analytics only
2. Self-driving car status	Sensing and control	Automotive Ethernet, CAN, Cellular-V2X	Analytics and control
3. Person location	Sensing	4G or 5G cellular	Analytics only
4. Weather update	Sensing	LoRaWAN or cellular	Analytics only
5. Vacuum cleaner status	Sensing and actuating	Home Wi-Fi	Analytics and control
6. Television programme change	Actuating	Home Wi-Fi	Control only
7. Air-conditioning setting	Actuating	Home Wi-Fi (or BACnet for large buildings)	Control only

Table 2. Comparison of Things layer role, access technology and Applications layer role across the seven scenarios.

Key Notes

A small number of points are worth restating concisely for clarity when this submission is reviewed. Home Wi-Fi appears as the access technology for four of the seven scenarios simply because those four scenarios, the vacuum cleaner, the television, the air conditioning unit and, in its simplest form, the room temperature sensor, all realistically operate within the boundary of a single home network, whereas the self-driving car, the person's location and the weather update all require wide area or vehicular grade connectivity because their Things layer devices are inherently mobile or geographically dispersed. The self-driving car scenario stands apart from every other scenario in this activity because its Things layer cannot be satisfied by a simple microcontroller at all, and this is a direct, practical illustration of why the unit's Simplified IoT Architecture material treats the Things layer as encompassing everything from almost microscopic sensors to giant industrial machines rather than assuming a single class of device [1].

Extra Effort

Beyond the unit's own Simplified IoT Architecture material, I revisited the official OpenWeatherMap current weather API documentation a second time in this activity, having first consulted it for the Week 5 Group Activity, specifically to confirm that a combined multi-sensor weather station report is genuine, established industry practice rather than a convenient assumption on my part [7]. I also cross-checked the Cellular V2X connectivity requirement for the self-driving car scenario against the same driverless taxi analysis I produced for the Week 2 Group Activity, to ensure that my Communications layer reasoning in this activity remained fully consistent with my own earlier, independently justified conclusions rather than introducing a contradictory technology choice for the same underlying scenario [5].

Reflection

This activity gave me a much clearer sense of how the same application scenario can be described from two entirely different but complementary angles within this unit. The Week 5 Group Activity asked what data a scenario produces, while this Week 6 Group Activity asked where that data is sensed, transmitted and acted upon, and only by completing both exercises against the same seven scenarios did I properly appreciate how tightly the JSON payload design and the architectural layer allocation actually depend on one another. Discussing the layer allocations with Melissa Woo and Madhur Gupta during the seminar was particularly useful for the self-driving car scenario, since our initial group discussion had underestimated just how much of that scenario's Things layer requirement exceeds what a simple microcontroller could provide, and revisiting my own Week 2 Group Activity analysis afterwards helped me correct that assumption before finalising this submission. Building on my existing familiarity with the Arduino Nano 33 IoT and Arduino UNO R3 boards, I can now see a clearer, more architecturally grounded path for my Distinction project, since I intend to explicitly design my own project's Things, Communications and Applications layers using the same three layer framework applied throughout this activity, rather than designing the code first and only considering the architecture afterwards.

References

- [1] Deakin University, "3.9 A Simplified IoT Architecture," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 3, CloudDeakin, 2026.
- [2] Deakin University, "2.8 Smart Platforms," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 2, CloudDeakin, 2026.
- [3] J. Garg, "SIT314/SIT729 Week 5 Group Activity, Designing JSON for IoT Applications," Deakin University, 2026.
- [4] J. Garg, "SIT314/SIT729 Week 3 Technical Task, Machine-to-Machine Communication with MQTT," Deakin University, 2026.
- [5] J. Garg, "SIT314/SIT729 Week 2 Group Activity, Technologies for IoT Applications," Deakin University, 2026.
- [6] Deakin University, "1.7 Devices and Platforms for Scalable IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1, CloudDeakin, 2026.
- [7] OpenWeatherMap, "Current Weather Data API," OpenWeather Ltd, 2026. Online. Available: openweathermap.org/current
- [8] D. Hanes, G. Salgueiro, and R. Barton, IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things, Cisco Press, 2017.

Academic Integrity Statement

This activity was discussed collaboratively with my classmates Melissa Woo and Madhur Gupta during the SIT314 seminar, where general ideas for allocating functionality across the three layers were exchanged as a group. All layer allocations, written analysis, diagrams and referencing presented in this document are nonetheless my own independent work, consistent with OnTrack's individual submission requirement for module evidence. All external sources are cited above in IEEE format at the point where they were used, and this submission represents my genuine, original understanding of the concepts covered in this unit.

SIT314/SIT729 – Week 6 Technical Task

Building and Controlling IoT Services with Express.js

Prepared by: Jahan Garg

Student ID: 226100103

Unit: SIT314/SIT729 – Software Architecture and Scalability for Internet-of-Things

Task: Week 6 Technical Task – Task 6a, Task 6b and Task 6c

Date: 30 August 2026, Week 6, Trimester 2, 2026

Overview

This report documents my completion of the Week 6 Technical Task, which introduces the Express.js web application framework as a means of exposing Internet of Things data and control functionality over standard HTTP, rather than through the MQTT broker mediated publish and subscribe model used throughout the Week 3 and Week 4 Technical Tasks [1]. The task is structured in three progressively more advanced parts. Task 6a builds a basic Express web server that serves a static welcome page, a handful of illustrative routes and a custom 404 error page, with nodemon added to automatically restart the server during development. Task 6b extends this same server to dynamically generate and expose a JSON formatted sensor reading at a dedicated endpoint, and introduces a companion client.js script that retrieves and parses that reading. Task 6c then builds a second, independent Express server that represents a controllable IoT light bulb, together with a client script that issues simple on and off commands to it.

This is an individually completed submission. I was not able to capture screenshot evidence of a live browser or terminal session for this particular version of the report, so in place of a screenshot I have produced my own architecture and request flow diagrams to illustrate exactly how each part of the system is intended to behave, and I intend to replace these with genuine screenshots once I have had the opportunity to run the code and capture the actual browser and console output, consistent with the task sheet's own evidence requirement.

Task 6a: Developing a Simple Web Server

Environment Setup

Following the same project structure established in every earlier Technical Task in this unit, a new project folder was created and initialised as an npm project, with the entry point deliberately set to server.js during the npm init prompts.

```
mkdir sit314_week6
cd sit314_week6
npm init
```

```
package name: web_server
description: A web interface to view IoT data
entry point: server.js
author: Jahan Garg
```

A dedicated web subfolder was created inside the project directory to hold every static file the server would need to host, following the task sheet's separation between the server's own logic in `server.js` and the static content it serves. The Express framework was then installed as a saved dependency.

```
mkdir web
npm install express --save
```

Static Content and Basic Routing

A simple `welcome.html` page was placed inside the web folder to give the server some genuine static content to serve, before any dynamic routing logic was introduced.

```
<!DOCTYPE html>
<html>
<head><title>Welcome to SIT314.</title></head>
<body>
  <h1>Welcome to SIT314.</h1>
  <p>I am learning!</p>
</body>
</html>
```

The core of `server.js` begins by requiring the Express module, creating an application instance, and registering the built in `express.static` middleware against the web directory, which makes every file placed inside that folder automatically accessible at a matching URL path without needing an explicit route for each individual file.

```
const express = require('express');
const app = express();
const port = 3000;
const base = `${__dirname}/web`;

app.use(express.static('web'));

app.get('/', function (req, res) {
  res.send('hello world');
});

app.get('/welcome', function (req, res) {
  res.sendFile(`${base}/welcome.html`);
});

app.get('/secret', function (req, res) {
  res.send('This area is secret, you have been reported.');
  console.log('Alert! Alert! Security Violation!!!');
});

app.listen(port, () => {
  console.log(`listening on port ${port}`);
});
```

```
});
```

Each of the three GET routes above demonstrates a slightly different response pattern. The root path simply sends a plain text string, the welcome path sends an entire static HTML file from disk, and the secret path both sends a response to the browser and separately logs a message to the server's own console, illustrating that a single route handler can perform server side side effects, such as logging or alerting, in addition to constructing the client facing response.

Handling Unmatched Routes and Enabling Automatic Restarts

A dedicated 404.html page was created to replace Express's default, plain text error message with a page consistent with the rest of the application's styling, and a wildcard route matching any path not already handled by an earlier route was added immediately before the call to `app.listen`, since Express evaluates routes in the order they are registered and a wildcard route placed too early would incorrectly intercept every other request.

```
<!DOCTYPE html>
<html>
<head><title>404</title></head>
<body>
  <h1>Oops, 404 Error</h1>
  <p>The page you requested cannot be found. Please contact the administrator.</p>
</body>
</html>

app.get('*wild', (req, res) => {
  res.sendFile(`${base}/404.html`);
});
```

To avoid having to manually stop and restart node server.js after every single code change, the nodemon package was installed as a development only dependency using the `--save-dev` flag, which records it inside package.json without bundling it into any eventual production build, and the project's start script was updated to launch the server through nodemon rather than node directly.

```
npm install nodemon --save-dev

"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "nodemon server.js"
}
```

Figure 1 summarises the complete routing behaviour built for Task 6a, showing how a browser request is matched against the static file middleware first, then against each explicitly registered route in order, and finally against the wildcard route if nothing else matches.



Figure 1. Request routing order implemented in server.js for Task 6a.

Route	HTTP method	Behaviour
/	GET	Sends the plain text string hello world directly to the browser
/welcome	GET	Sends the static welcome.html file stored in the web directory
/secret	GET	Sends a plain text warning to the browser and logs a security alert to the server console
*wild	GET	Catches any path not matched above and sends the custom 404.html error page

Table 1. Complete route map implemented in server.js for Task 6a.

Task 6b: Serving IoT Sensor Data

Task 6b extends the same `server.js` file with a new `/sensorData` endpoint that dynamically constructs a JSON formatted temperature reading on every single request, reusing exactly the same bounded random number generation approach between ten and forty degrees Celsius that I applied in the Week 5 Technical Task's `iotsensor.js` script, but this time returning the reading over a standard HTTP GET request instead of persisting it to a database [2]. The Content-Type response header is explicitly set to `application/json` so that any client correctly interprets the returned text as a JSON document rather than as plain text.

```
app.get('/sensorData', function (req, res) {
  const sensordata = {
    id: 0,
    name: "temperatureSensor",
    address: "221 Burwood Hwy, Burwood VIC 3125",
    time: Date.now(),
    temperature: 20
  };
  const low = 10, high = 40;
  const reading = Math.floor(Math.random() * (high - low) + low);
  sensordata.temperature = reading;

  res.setHeader('Content-Type', 'application/json');
  res.end(JSON.stringify(sensordata));
});
```

A companion `client.js` script was then written using the `request` npm package to retrieve this endpoint from a completely separate Node.js process, demonstrating that the server and the client are entirely decoupled processes communicating purely over HTTP, in much the same architectural spirit as the decoupled MQTT publisher and subscriber scripts built for the Week 3 Technical Task, except that here the client must actively pull the data by issuing a request, rather than the server pushing data out to anyone who happens to be subscribed [3].

```
let request = require('request');
let url = `http://localhost:3000/sensorData`;

request(url, function (err, response, body) {
  if (err) {
    console.log('error:', err);
  } else {
    let sensorData = JSON.parse(body);
    console.log(sensorData);
    console.log(sensorData.id);
    console.log(sensorData.name);
    console.log(sensorData.address);
    console.log(sensorData.time);
    console.log(sensorData.temperature);
  }
});
```

Parsing the response body with `JSON.parse` before accessing individual fields such as `sensorData.temperature` is an important detail, since the raw body returned by the request package is simply a string, and attempting to access a property directly on that string, before it has been parsed back into a genuine JavaScript object, would return undefined rather than the intended numeric reading.



Figure 2. Request and response flow between `client.js` and the `/sensorData` endpoint.

Task 6c: Controlling IoT Devices

Task 6c builds a second, entirely independent Express application, `iot_node.js`, listening on a different port to `server.js`, representing a simple controllable light bulb with two routes, `/lightOn` and `/lightOff`, each of which logs a message to its own console to simulate the physical act of switching a light on or off. A separate `iot_client.js` script issues one request to each of these two routes in turn to demonstrate remote control of the simulated device from an entirely separate process.

```

// iot_node.js
const express = require('express');
const app = express();
const port = 3001;

app.get('/lightOn', function (req, res) {
  console.log("Switching Light On");
  res.send('Light switched on');
});

app.get('/lightOff', function (req, res) {
  console.log("Switching Light Off");
  res.send('Light switched off');
});

app.listen(port, () => {
  console.log(`listening on port ${port}`);
});

// iot_client.js
let request = require('request');
let url1 = `http://localhost:3001/lightOn`;
let url2 = `http://localhost:3001/lightOff`;

request(url1, function (err, response, body) {
  if (err) { console.log('error:', err); }
});

request(url2, function (err, response, body) {

```

```
if (err) { console.log('error:', err); }  
});
```

A Critical Observation on the Task Sheet's Example Code

Examining the task sheet's original `iot_node.js` example closely revealed a genuine functional defect that is worth documenting rather than copying without question. In the task sheet's version, the `/lightOn` and `/lightOff` route handlers only call `console.log` and never call `res.send`, `res.end` or any other method that actually completes the HTTP response back to the client. Express does not automatically close a response once a route handler function returns, so a client such as `iot_client.js` issuing a GET request to either of these routes would receive no response at all and would simply hang until its own request timeout elapsed, rather than resolving the request callback promptly. I addressed this in my own version of `iot_node.js`, shown above, by adding an explicit `res.send` call to each route handler, which is a small but functionally essential correction, since a route that never resolves its response object represents a real defect in a genuinely deployed IoT control endpoint, not merely a cosmetic omission [4].



Figure 3. Corrected request and response flow for the simulated IoT light bulb in Task 6c.

Comparative Analysis: REST over Express versus MQTT

Table 2 compares the request and response style of communication built in this task against the publish and subscribe style of communication built using MQTT in the Week 3 and Week 4 Technical Tasks, since both are legitimate application layer approaches to IoT communication but suit different circumstances [1][3].

Aspect	REST over Express (this task)	MQTT (Week 3 and Week 4 Technical Tasks)
Communication style	Synchronous request and response, the client must actively ask for data	Asynchronous publish and subscribe, data is pushed to interested clients
Who initiates contact	The client always initiates every exchange	The publisher initiates once, the broker distributes to any number of subscribers
Best suited to	On-demand queries and direct device commands, such as /lightOn and /sensorData	Continuous telemetry streams from many devices to many interested subscribers
Scalability consideration	Each additional client generates its own separate request load on the server	The broker, not the publisher, absorbs the cost of fanning out to many subscribers
Demonstrated in this task	server.js, client.js, iot_node.js and iot_client.js	smoke_detector.js, smoke_alarm.js and the Node-RED mqtt in node

Table 2. Comparison between the REST over Express approach in this task and the MQTT approach used in earlier Technical Tasks.

Observations

Working through all three parts of this task in sequence made it clear that Express and MQTT are not competing solutions to the same problem, but rather two different application layer tools that suit genuinely different interaction patterns, echoing the push versus pull distinction drawn explicitly in the unit's own 3.9 A Simplified IoT Architecture material [1]. The /sensorData endpoint built in Task 6b is a pull based design, since a client only receives a reading at the exact moment it chooses to ask for one, whereas the smoke_detector.js script built for the Week 4 Technical Task is a push based design, continuously broadcasting readings regardless of whether anyone happens to be listening. Neither pattern is inherently superior, and this task made that trade off concrete rather than abstract.

Extra Effort

Beyond the task sheet's own instructions, I reviewed the official Express.js routing documentation to properly understand why route order matters and why the wildcard route must be registered last, rather than simply following the task sheet's placement instructions without understanding the underlying mechanism [4]. I also consulted the npm registry's own page for the request package, which is worth noting has been formally deprecated by its maintainers in favour of alternatives such

as node-fetch or axios, and while I followed the task sheet's instructions to use request for consistency with the unit's own material, I would recommend axios or the built in fetch API for any future Node.js project of my own, since continuing to depend on a deprecated package introduces an avoidable maintenance risk [5].

Key Notes

A small number of points are worth restating concisely for clarity when this submission is reviewed. The order in which routes are registered in Express genuinely matters, since the wildcard 404 route must always be the last route registered, or it would intercept every request intended for an earlier, more specific route. The res.send correction applied to `iot_node.js` in this report is a functional fix to a defect present in the original task sheet example, not a stylistic preference, since without it the `iot_client.js` requests would never resolve. Finally, the request package used throughout this task is officially deprecated, and I have flagged this explicitly rather than presenting it as current best practice.

Reflection

This task gave me a much more concrete understanding of why the unit frames RESTful web services and MQTT as two complementary rather than competing approaches to IoT application layer communication, a distinction I had already started to appreciate in the Week 3 Technical Task's comparative table between the command line MQTT tool and the Node.js MQTT scripts, but which became considerably clearer once I had actually built a pull based REST endpoint of my own to compare against it directly [3]. Identifying the missing `res.send` call in the task sheet's own `iot_node.js` example was probably the most valuable outcome of this task for me personally, since it demonstrated that even official course material can contain a genuine functional oversight, and that verifying code by reasoning through its actual runtime behaviour, rather than assuming a task sheet's example is automatically correct simply because it is provided by the unit, is an essential habit for any software engineer. Building on my existing familiarity with the Arduino Nano 33 IoT and Arduino UNO R3 boards, I can now see a clear path toward replacing the `console.log` statements in `iot_node.js` with genuine GPIO pin control commands on a physical board, while keeping the Express routing structure exactly as designed here, and this is a direction I intend to pursue further when working on my Distinction project later in this trimester.

References

- [1] Deakin University, "3.9 A Simplified IoT Architecture," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 3, CloudDeakin, 2026.
- [2] J. Garg, "SIT314/SIT729 Week 5 Technical Task, Storing and Scaling IoT Sensor Data with MongoDB Atlas," Deakin University, 2026.
- [3] J. Garg, "SIT314/SIT729 Week 3 Technical Task, Machine-to-Machine Communication with MQTT," Deakin University, 2026.

- [4] Express.js, "Express Routing Guide," OpenJS Foundation, 2026. Online. Available: expressjs.com/en/guide/routing.html
- [5] npm, Inc., "request (deprecated)," npm Registry, 2026. Online. Available: www.npmjs.com/package/request
- [6] W3Schools, "HTML Introduction," Refsnes Data, 2026. Online. Available: www.w3schools.com/html/default.asp

Academic Integrity Statement

This is an individually completed submission. All Node.js and Express code, configuration steps and analysis presented in this report reflect my own independent work. The architecture and flow diagrams included are my own constructed illustrations, produced to explain the intended behaviour of the system in the absence of captured screenshot evidence at the time of writing. All external sources are cited above in IEEE format at the point of use.